

멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 지원 기술 연구



저자 1 : 202113111 김재우 projw1024@pusan.ac.kr

저자 2 : 202155578 이상현 ask0208ask@pusan.ac.kr

저자 3 : 202055625 황수환 tngghks2866@pusan.ac.kr

지도교수 : 염근혁

목 차

1. 서론	1
1.1. 연구 배경	1
1.2. 기존 문제점	1
1.2.1. LLM 환각 현상에 따른 산출물 오류	1
1.2.2. 클라우드 리소스 의존 관계성	2
1.2.3. AI 에이전트 의도 전파 과정의 불명확성	2
1.3. 연구 목표	2
2. 이론적 배경 및 관련 기술	3
2.1. 클라우드 네이티브 애플리케이션	3
2.2. LLM 및 AI 에이전트 기술	4
2.2.1. 대규모 언어 모델(LLM)	4
2.2.2. AI 에이전트	5
2.2.3. LangGraph	5
2.2.4. OpenHands SDK	6
2.2.5. MetaGPT	6
2.2.6. ChatDev	7
2.2.7. BERT	7
2.3. 요구사항·설계 명세 기술	8
2.3.1. BCE 설계 기법	8
2.3.2. UML	9
2.3.3. OpenAPI Specification	9
2.4. 컨테이너·IaC 기술 및 클라우드 플랫폼	9
2.4.1. Docker	9
2.4.2. Terraform	9
2.4.3. OpenTofu	10
2.4.4. Amazon Web Services(AWS)	11
2.4.5. Microsoft Azure	11
2.4.6. Google Cloud Platform(GCP)	12
3. 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 지원 방법	13

3.1. 기술 개요	13
3.2. 클라우드 리소스 분석	14
3.3. 요구사항 분석 에이전트	19
3.4. 설계 에이전트	21
3.5. 구현 에이전트	23
3.6. 테스트 에이전트	23
4. 사례 애플리케이션 생성 및 AWS 배포 사례.....	25
4.1. 요구사항 분석 에이전트	25
4.2. 시스템 설계 에이전트	31
4.3. 시스템 구현 에이전트	44
4.4. 테스트 에이전트	52
4.5. 클라우드 리소스 배포	55
5. 비교 분석 및 평가	59
5.1. 평가 대상과 방법	59
5.1.1. 평가 대상 및 비교 조건	59
5.1.2. 평가 항목 및 선정 근거	59
5.2. 평가 결과	60
6. 결론 및 향후 연구 방향.....	63
7. 참고 문헌	64

1. 서론

1.1. 연구 배경

클라우드 네이티브 애플리케이션은 클라우드 리소스 기반으로 설계-배포-관리되는 서비스 유형을 말한다. 클라우드 네이티브 애플리케이션은 서비스 동작 환경 설계, 클라우드 리소스(네트워크, 가상머신 등) 배포, 애플리케이션 관리로 이어지는 구조를 지원하는 것이 중요하다. 이에, AI 에이전트 기술을 바탕으로 다양한 유형의 컨텍스트를 통해 서비스 개발을 지원하는 방법이 주로 활용되고 있다.

클라우드 네이티브 애플리케이션 개발 시 AI 에이전트를 활용하는 경우 다음과 같은 기술적 보완이 필요하다. 첫째, AI 에이전트가 사용자 의도를 잘못 해석하는 환각(Hallucination) 현상을 보완해야 한다. 환각 현상이 발생하면, 사용자가 요구하지 않은 클라우드 리소스와 서비스 기능이 산출물에 포함된다. 둘째, 클라우드 리소스 간 의존성에 따른 클라우드 운영 제약사항을 해소해야 한다. 클라우드 리소스는 CSP 별 리소스 명세에 따라 선택이 강제될 수 있으며, 사용자 요구사항(비용, 성능 등)에 따른 가이드라인 학습이 필요하다. 셋째, 클라우드 네이티브 애플리케이션 개발 과정에 초점을 둔 지원 기술이 필요하다. 클라우드 네이티브 애플리케이션은 소프트웨어 개발 과정을 고려한 클라우드 리소스를 개발 단계별로 배포 및 연계해야 한다.

따라서 본 과제에서는 클라우드 리소스와 클라우드 네이티브 서비스의 생성, 배포, 실행을 지원하는 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 지원 기술을 제안한다. 제안하는 기술은 1) 요구사항 기반 클라우드 네이티브 애플리케이션 검증 및 멀티 AI 에이전트 구조 설계, 2) 클라우드 네이티브 애플리케이션 제약사항 해소 방법 도출, 3) 멀티 AI 에이전트 간 산출물 연계 방안 구축을 목표로 한다.

1.2. 기존 문제점

1.2.1. LLM 환각 현상에 따른 산출물 오류

LLM 의 환각(Hallucination)은 문법적으로 자연스러워 보이지만, 실제 근거가 없거나 입력된 요구사항과 모순되는 정보를 생성하는 현상이다[1]. 소프트웨어 개발에서 이러한 환각은 개발자가 요청하지 않은 기능이나 변경 사항이 코드에 포함되는 형태로 나타날 수 있다. 예를 들어, 단순한 오류 수정이나 화면 요소 변경을 요청했음에도 LLM 이 데이터베이스 구조를 수정하거나, 새로운 권한 처리·외부 API 호출·데이터 저장 기능을 임의로 추가할 수 있다. 또한 기존 기능의 동작 방식을 충분히 고려하지 않고 관련 없는

모듈까지 수정하여, 의도하지 않은 기능 변경이나 기존 기능의 오류를 초래할 수 있다.

1.2.2. 클라우드 리소스 의존 관계성

클라우드 네이티브 애플리케이션이 동작하는 클라우드 환경은 다양한 클라우드 리소스를 연결 · 참조하여 형성된다. 클라우드 리소스는 생성 시 선행 배포가 필요한 리소스가 존재하며, 여러 CSP(Cloud Service Provider)를 동시에 제어하는 환경에서는 CSP 별로 다른 리소스 관리 방법이 요구된다. 즉, CSP 마다 생성하고자 하는 클라우드 리소스를 배포하고 클라우드 네이티브 애플리케이션에 맞춰 클라우드 환경을 구성해야 하는 문제가 있다.

1.2.3. AI 에이전트 의도 전파 과정의 불명확성

AI 에이전트의 의도 전파란 사용자의 목표가 서로 다른 에이전트에 연쇄 전달되는 과정을 말한다. AI 에이전트는 의도 전파를 통해 사용자의 목표를 공유하며, 사용자의 목표를 달성할 수 있도록 동작한다. 사용자의 목표가 부정확하게 작성된 경우, 사용자의 의도는 다른 에이전트를 거치면서 결함요소로 동작한다. 즉, 의도가 변질되어 사용자가 목표로 하는 최종 산출물에 결함이 발생할 수 있다.

1.3. 연구 목표

본 연구는 멀티 AI 에이전트 기반으로 클라우드 네이티브 애플리케이션의 자연어 요구사항을 분석하고, UML 다이어그램으로 설계하며, IaC·소스 코드를 통해 시스템을 구현, 산출물 테스트를 수행하는 기술을 연구하는 것이다.

- 1) **멀티 AI 에이전트 구조 설계:** 요구사항 분석, 시스템 설계, 구현과 테스트를 담당하는 에이전트를 구성하고, 각 단계의 구조화된 산출물이 다음 단계의 입력으로 이어지도록 연계한다.
- 2) **클라우드 네이티브 애플리케이션 제약사항 해소 방법:** AWS·Azure·GCP 리소스 간 의존 관계, VM 사양, 비용과 성능 정보를 활용하여 사용자 요구조건에 맞는 클라우드 리소스 구성과 배포 방안을 제시한다.
- 3) **멀티 AI 에이전트 간 산출물 연계 방안 구축:** 사용자가 단계별 산출물을 검토하고 피드백을 제공할 수 있도록 하며, 피드백의 대상과 영향 범위에 따라 관련 산출물을 일관되게 수정한다.

2. 이론적 배경 및 관련 기술

2.1. 클라우드 네이티브 애플리케이션

클라우드 네이티브 애플리케이션[2]은 클라우드 환경의 유연한 자원 활용을 바탕으로 수요 변화와 장애에 대응하고, 애플리케이션을 지속적으로 배포하고 운영할 수 있도록 설계된 애플리케이션이다.

주요 특징으로는 컨테이너를 이용한 실행 환경의 표준화, 서비스를 독립적으로 배포하고 확장할 수 있는 구조, 자동화를 통한 인프라와 배포 관리, 그리고 시스템의 상태와 문제를 확인하기 위한 관찰 가능성(observability)이 있다. 인프라와 배포를 관리할 때에는 원하는 시스템의 상태를 미리 정의하고, 관리 시스템이 실제 상태를 이에 맞게 자동으로 유지하도록 하는 선언형 방식을 사용할 수 있다. 또한 로그, 메트릭, 트레이스 등의 정보를 수집하여 애플리케이션의 동작 상태와 장애 원인을 파악할 수 있다.

이러한 방식을 적용하면 실행 환경에 따른 차이를 줄이고, 인프라와 애플리케이션의 변경 사항을 반복 가능한 절차를 통해 배포할 수 있다. 또한 서비스가 독립적인 단위로 구성된 경우 필요한 부분만 개별적으로 배포하거나 확장할 수 있으며, 자동화된 복구와 배포 과정을 통해 변화가 잦은 환경에서도 안정적으로 서비스를 운영할 수 있다.

Cloud Native Reference Architecture

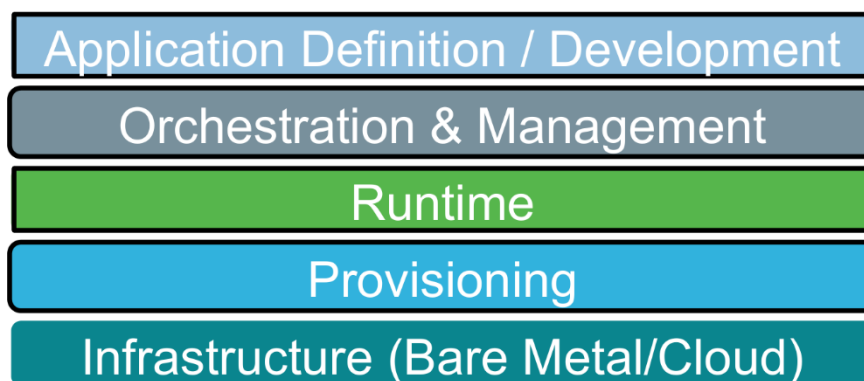


그림 1. CNCF 클라우드 네이티브 참조 구조의 계층[3]

그림 1 은 클라우드 네이티브 환경을 인프라, 프로비저닝, 런타임, 오케스트레이션·관리와 애플리케이션 정의·개발 계층으로 구분한다. 아래 계층의 컴퓨팅·네트워크·저장소 위에 리소스 생성과 실행 환경이 놓이고, 그 위에서 애플리케이션의 배치·확장·복구와 개발이 이루어진다. 이 구조는 클라우드 네이티브 애플리케이션이 하나의 도구로 구현되는 것이 아니라 인프라 준비, 실행 환경 관리, 배포·운영과 애플리케이션 개발이 함께 연계되어야 함을 보여 준다. 이러한 특성으로 인해 클라우드 네이티브 애플리케이션을 개발하기 위해서는 애플리케이션뿐만 아니라 이를 실행하기 위한 클라우드 리소스와 배포 환경을 함께 고려할 필요가 있다.

2.2. LLM 및 AI 에이전트 기술

2.2.1. 대규모 언어 모델(LLM)

대규모 언어 모델(Large Language Model, LLM)[4]은 방대한 양의 텍스트를 학습하여 주어진 입력을 바탕으로 다음에 등장할 토큰을 예측하고, 이를 반복하여 텍스트를 생성하는 인공지능 모델이다. 문장이 입력되면 먼저 모델이 처리할 수 있는 단위인 토큰(token)으로 나눈다. 토큰은 단어 전체나 단어의 일부 또는 문장 부호 등에 해당하며, 각 토큰은 모델이 계산에 사용할 수 있는 벡터로 변환된다.

LLM 은 주로 Transformer 구조를 기반으로 한다. Transformer 는 여러 토큰 사이의 관계를 동시에 계산할 수 있도록 설계된 신경망 구조이다. 특히 Self-Attention 을 이용하여 각 토큰이 문맥 내의 다른 토큰과 어떤 관계를 가지는지를 계산함으로써, 문장에서 서로 떨어져 있는 단어들 사이의 관계도 반영할 수 있다.

변환된 토큰 벡터는 Transformer 의 여러 계층을 통과한다. 각 계층에서는 Self-Attention 을 통해 하나의 토큰을 처리할 때 앞서 등장한 다른 토큰들을 얼마나 참고해야 하는지를 계산한다. 토큰 간의 관련성에 따라 서로 다른 중요도가 부여되며, 이를 반영하여 각 토큰의 벡터가 갱신된다. 이러한 처리를 여러 계층에서 반복하면서 각 토큰의 표현에는 입력 문맥의 관계가 반영된다. Transformer 의 처리가 끝나면 LLM 은 지금까지 입력된 토큰을 바탕으로 다음에 올 수 있는 토큰들의 확률을 계산하고 그중 하나를 선택한다. 선택된 토큰을 기존 입력에 추가한 뒤 동일한 과정을 반복함으로써 최종적으로 텍스트를 생성한다.

이러한 자연어 이해 및 생성 능력은 사용자의 요구사항을 분석하고 소프트웨어 개발에 필요한 산출물을 생성하는 데 활용할 수 있다.

2.2.2. AI 에이전트

AI 에이전트[5]는 LLM 을 활용하여 사용자의 요청을 해석하고 수행해야 할 작업 목표를 파악한 뒤, 현재 상태를 바탕으로 목표 달성을 위한 행동을 결정하고 필요한 외부 도구를 호출하는 소프트웨어 시스템이다. 사용자의 요청이 입력되면 에이전트는 이를 바탕으로 필요한 작업을 여러 단계로 나누고, 각 단계에서 필요한 정보를 확인한 뒤 검색, 파일 편집, 명령 실행과 같은 적절한 도구를 선택하여 실행한다. 도구의 실행 결과는 이후의 판단에 다시 반영되며, 목표가 달성되거나 작업 종료 조건을 만족할 때까지 이러한 과정이 반복된다.

이전 입력, 도구 실행 결과 및 현재 작업 상태는 메모리에 유지되어 이후 단계에서 다음 행동을 결정하는 근거로 활용될 수 있다. 또한 여러 AI 에이전트를 함께 사용하는 멀티 에이전트 시스템에서는 각 에이전트에 서로 다른 역할을 부여하여 복잡한 작업을 분담할 수 있다. 예를 들어 요구사항 분석, 시스템 설계, 구현과 같이 역할을 구분하고, 한 에이전트가 생성한 산출물을 다음 에이전트의 입력으로 전달함으로써 여러 단계의 작업을 연계하여 수행할 수 있다. 특히 소프트웨어 개발과 같이 여러 작업이 연속적으로 수행되는 과정에서는 역할별 에이전트가 작업을 분담하고 결과를 다음 작업으로 전달하는 형태로 활용할 수 있다.

2.2.3. LangGraph

LangGraph[6]는 AI 에이전트의 작업 과정을 그래프 형태로 구성하고 실행할 수 있도록 지원하는 프레임워크이다. 그래프는 작업을 수행하는 단위인 노드(node)와 노드 사이의 실행 관계를 나타내는 엣지(edge)로 구성된다. 각 노드는 LLM 호출, 도구 실행, 데이터 처리와 같은 특정 작업을 담당하며, 엣지는 한 노드의 작업이 끝난 뒤 어떤 노드를 실행할지를 연결한다.

작업이 진행되는 동안 필요한 입력, 중간 결과 및 현재 진행 정보는 상태(state)로 관리된다. 각 노드는 이 상태를 전달받아 작업을 수행하고, 처리 결과를 다시 상태에 반영한다. 또한 조건부 엣지(conditional edge)를 사용하면 현재 상태나 처리 결과에 따라 다음에 실행할 노드를 선택할 수 있다. 이를 통해 순차적인 작업뿐만 아니라 조건에 따른 분기와 반복을 포함하는 AI 에이전트의 작업 흐름을 구성할 수 있다. 이와 같은 상태 관리와 조건부 실행은 여러 단계로 구성되는 AI 에이전트의 작업 흐름을 제어하는데 유용하다.

2.2.4. OpenHands SDK

OpenHands Software Agent SDK[7]는 LLM 과 파일 편집, 명령 실행 등의 도구를 결합하여 소프트웨어 개발 작업을 수행하는 AI 에이전트를 구성할 수 있도록 지원하는 프레임워크이다. OpenHands SDK 의 주요 구성요소에는 Agent, Conversation, Tool, Workspace 가 있다. Agent 는 LLM 을 이용하여 사용자의 요청과 지금까지의 작업 이력을 분석하고 다음에 수행할 행동을 결정한다. Conversation 은 사용자와 Agent 사이의 작업 과정을 관리하며, 사용자 요청과 Agent 의 행동, 실행 결과 등의 작업 이력과 현재 실행 상태를 관리한다. Tool 은 파일 편집이나 명령 실행처럼 Agent 가 사용할 수 있는 기능을 제공한다. Workspace 는 Tool 을 통해 파일을 수정하거나 명령을 실행하는 등 실제 작업이 수행되는 실행 환경이다.

사용자의 작업 요청이 Conversation 에 전달되면 Agent 는 지금까지의 작업 이력을 바탕으로 LLM 을 호출하여 다음 행동을 결정한다. 파일을 수정하거나 명령을 실행해야 하는 경우 Agent 는 적절한 Tool 을 선택하여 Workspace 에서 작업을 수행한다.

Agent 가 생성한 도구 실행 요청은 ActionEvent 로 기록되고, 도구를 실행한 결과는 ObservationEvent 로 기록된다. 사용자 메시지를 포함한 이러한 기록은 모두 Event 형태로 Conversation 에 순서대로 저장된다. Agent 는 새롭게 추가된 실행 결과를 포함한 작업 이력을 다시 확인하여 코드를 추가로 수정하거나 검증 명령을 수행한다. Conversation 은 이러한 과정을 관리하며, Agent 가 작업을 완료할 때까지 판단과 도구 실행 과정이 반복된다. 이를 통해 AI 에이전트가 결과를 생성하는 것에 그치지 않고 파일 수정과 명령 실행을 포함한 실제 소프트웨어 개발 작업을 수행할 수 있다.

2.2.5. MetaGPT

MetaGPT[8]는 소프트웨어 개발 조직의 역할과 표준 운영 절차(Standard Operating Procedure, SOP)를 LLM 기반 멀티 에이전트의 작업 흐름에 적용한 프레임워크이다. 하나의 LLM 이 사용자의 요청을 바로 코드로 변환하는 대신, 제품 관리자(Product Manager), 아키텍트(Architect), 프로젝트 관리자(Project Manager), 엔지니어(Engineer) 등 서로 다른 역할을 담당하는 에이전트가 소프트웨어 개발 절차에 따라 작업을 분담한다.

제품 관리자 에이전트는 사용자의 요청을 분석하여 요구사항과 사용자 스토리 등을 포함한 요구사항 문서를 작성한다. 아키텍트 에이전트는 이를 바탕으로 시스템 구조, 데이터 구조 및 인터페이스 등의 설계를 수행하며, 프로젝트 관리자 에이전트는 설계 결과를 실제 구현에 필요한 세부 작업으로 나눈다. 엔지니어 에이전트는 이러한 작업과 설계를

바탕으로 코드를 작성하며, 품질 검증 단계에서는 생성된 코드와 실행 또는 테스트 결과를 확인한다.

MetaGPT 의 SOP 는 각 역할이 수행해야 할 작업과 산출물, 그리고 역할 사이의 작업 전달 절차를 정의한다. 이에 따라 한 역할에서 생성된 요구사항 문서, 시스템 설계, 작업 목록과 같은 산출물이 다음 역할의 입력으로 전달되며, 여러 에이전트가 정해진 개발 절차에 따라 하나의 소프트웨어 개발 작업을 수행한다. 이처럼 개발 역할을 분리하고 산출물을 단계적으로 전달하는 방식은 멀티 AI 에이전트를 이용한 소프트웨어 개발 지원 방법으로 활용되고 있다.

2.2.6. ChatDev

ChatDev[9]는 서로 다른 역할을 가진 LLM 기반 에이전트들이 대화를 통해 협력하여 소프트웨어 개발 과정을 수행하는 멀티 에이전트 프레임워크이다. 전체 개발 과정은 설계, 코딩, 테스트 등의 단계와 세부 작업으로 나뉘며, 각 작업에서 서로 다른 역할을 가진 두 에이전트가 대화를 통해 결과를 생성한다. 이러한 세부 작업은 정해진 순서에 따라 연결되며, 앞 단계에서 생성된 결과가 다음 단계의 작업에 활용된다.

각 하위 작업에서는 한 에이전트가 수행해야 할 작업과 요구사항을 제시하고, 다른 에이전트가 이에 따른 결과를 작성한다. 두 에이전트는 여러 차례 대화를 주고받으면서 결과를 수정하고 구체화한다. 이 과정에서 지시가 불명확하거나 필요한 정보가 부족하면 상대 에이전트에게 질문하여 내용을 확인하며, ChatDev 는 이러한 방식을 대화형 환각 완화(communitive dehallucination)라고 표현하였다. 이를 통해 잘못된 가정이나 정보 누락을 줄이고 작업 결과를 구체화하며, 각 작업에서 생성된 결과를 다음 작업으로 전달하여 전체 소프트웨어 개발 과정을 이어 간다. 이러한 에이전트 간 대화 방식은 개발 과정에서 발생할 수 있는 잘못된 가정이나 정보 누락을 줄이고 산출물을 구체화하는 데 활용된다.

2.2.7. BERT

BERT(Bidirectional Encoder Representations from Transformers)[10]는 문장의 앞뒤 문맥을 함께 고려하여 각 토큰의 표현을 학습하는 언어 모델이다. 입력 문장이 토큰으로 나뉘면 각 토큰은 벡터로 변환되어 여러 Transformer 인코더 계층을 통과한다. 각 인코더에서는 Self-Attention 을 통해 해당 토큰의 앞과 뒤에 위치한 다른 토큰과의 관계를 함께 계산한다. 이러한 과정을 반복함으로써 각 토큰은 문장 내의 문맥과 다른 토큰과의 관계가 반영된 벡터로 표현된다.

BERT의 사전 학습에서는 문장의 일부 토큰을 [MASK]로 가리고 주변 문맥을 이용하여 원래 토큰을 예측하는 마스크드 언어 모델(Masked Language Model, MLM)을 사용한다. 또한 원래 BERT 모델에서는 두 문장이 실제로 연속해서 등장한 문장인지를 판별하는 다음 문장 예측(Next Sentence Prediction, NSP)을 함께 사용한다. 이러한 학습 과정에서 모델의 가중치가 조정되면서 단어가 문맥에 따라 가지는 의미와 문장 사이의 관계를 학습한다.

문장 분류에 사용할 때에는 사전 학습된 가중치를 정답 범주가 표시된 라벨 데이터에 맞춰 파인 튜닝한다. 문장 맨 앞에는 전체 문장을 대표하는 CLS(Classification) 토큰을 넣고, Transformer 인코더를 지난 CLS의 벡터를 출력 계층에 전달한다. 출력 계층은 이 벡터를 분류 대상인 각 클래스(class)의 확률로 바꾸어 최종 결과를 결정한다.

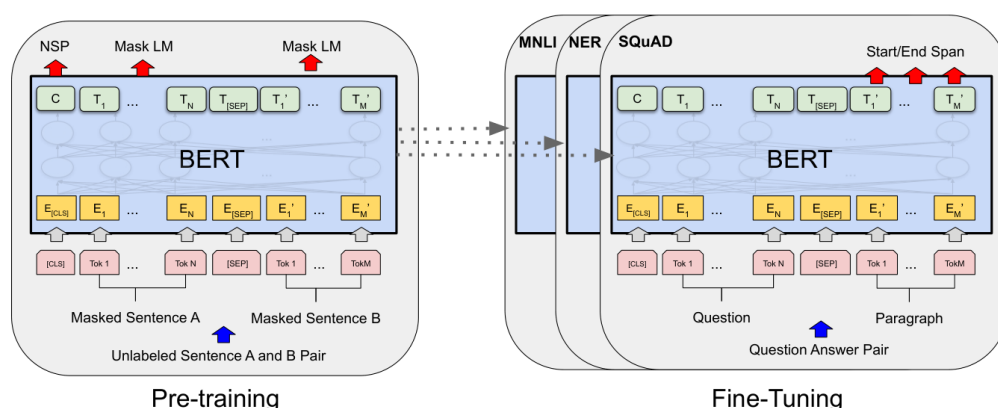


그림 2. BERT의 사전 학습과 파인 튜닝 구조[10]

그림 2의 왼쪽은 라벨이 없는 텍스트 데이터로 BERT를 사전 학습하는 과정을, 오른쪽은 사전 학습된 모델에 작업별 출력 계층을 연결하고 라벨 데이터로 파인 튜닝하는 과정이다. 사전 학습에서 얻은 언어 표현을 문장 분류나 질의응답에 맞게 조정하여 사용한다. 문맥을 반영한 문장 표현과 분류 능력을 바탕으로 자연어로 작성된 요구사항의 유형을 구분하는 데 활용할 수 있다.

2.3. 요구사항·설계 명세 기술

2.3.1. BCE 설계 기법

BCE(Boundary-Control-Entity)[11]는 유스케이스 분석 과정에서 클래스를 Boundary, Control, Entity로 구분하는 객체지향 분석·설계 기법이다. Boundary는 사용자나 외부 시스템과의 상호작용을 담당하고, Control은 유스케이스 수행에 필요한 처리 흐름과 객체

간 협력을 조정하며, Entity 는 여러 유스케이스에서 공유되는 도메인 정보와 상태를 표현한다. 이러한 역할 구분은 요구사항으로부터 클래스를 구분하고 시스템의 주요 구성 요소를 도출하는 데 활용할 수 있다.

2.3.2. UML

UML(Unified Modeling Language)[12]은 OMG(Object Management Group)가 표준화한 범용 시각적 모델링 언어로, 소프트웨어 시스템의 구조와 동작을 명세하고 표현하기 위한 언어이다. UML 의 모델은 시스템의 구성 요소와 관계, 동작을 추상화한 표현이며, 다이어그램은 이러한 모델의 특정 측면을 시각적으로 나타낸 것이다. UML 은 시스템을 구성하는 요소들의 정적 구조뿐 아니라 시간에 따른 상태 변화와 요소 간 상호작용 등의 동적 동작을 표현한다. 또한 시스템 전체의 구성부터 개별 요소의 세부 동작까지 서로 다른 추상화 수준에서 모델을 기술할 수 있다. 이를 통해 자연어로 표현된 요구사항과 시스템의 구조 및 동작을 개발 과정에서 활용 가능한 형태로 명세할 수 있다.

2.3.3. OpenAPI Specification

OpenAPI Specification(OAS)[13]은 HTTP API 의 기능과 호출 규칙을 기술하기 위한 표준 인터페이스 명세이다. 특정 프로그래밍 언어에 종속되지 않으며, 서비스의 소스 코드나 실제 통신 내용을 확인하지 않고도 API 의 외부 인터페이스를 파악할 수 있도록 정의된다. 따라서 설계된 기능의 API 구조와 호출 규칙을 명확한 형식으로 정의하고 이후 개발 과정에 전달하는 데 활용할 수 있다.

2.4. 컨테이너·IaC 기술 및 클라우드 플랫폼

2.4.1. Docker

Docker[14]는 애플리케이션을 컨테이너 단위로 패키징하고 배포·실행하는 플랫폼이다. 컨테이너는 애플리케이션과 실행에 필요한 라이브러리 및 의존성을 포함하는 격리된 실행 환경이며, 이미지는 이러한 컨테이너를 생성하기 위한 읽기 전용 템플릿이다. 하나의 이미지에서 여러 컨테이너를 생성할 수 있으며, 각 컨테이너는 이미지와 실행 시 지정된 설정을 바탕으로 동작한다. 특히 클라우드 네이티브 환경에서는 애플리케이션과 실행에 필요한 의존성을 함께 패키징하여 일관된 실행 환경을 구성하는 데 유용하다.

2.4.2. Terraform

Terraform[23]은 클라우드와 외부 서비스의 리소스를 선언형 구성으로 관리하는 IaC 도구이다. 일반적인 Terraform 실행은 provider 플러그인과 모듈을 준비하는 init,

구성·현재 상태·실제 리소스의 차이를 계산하는 plan, 승인한 계획을 인프라에 반영하는 apply 순서로 진행한다. 상태 파일은 구성의 리소스 인스턴스와 실제 원격 객체를 연결하며 의존성 메타데이터도 보존한다.

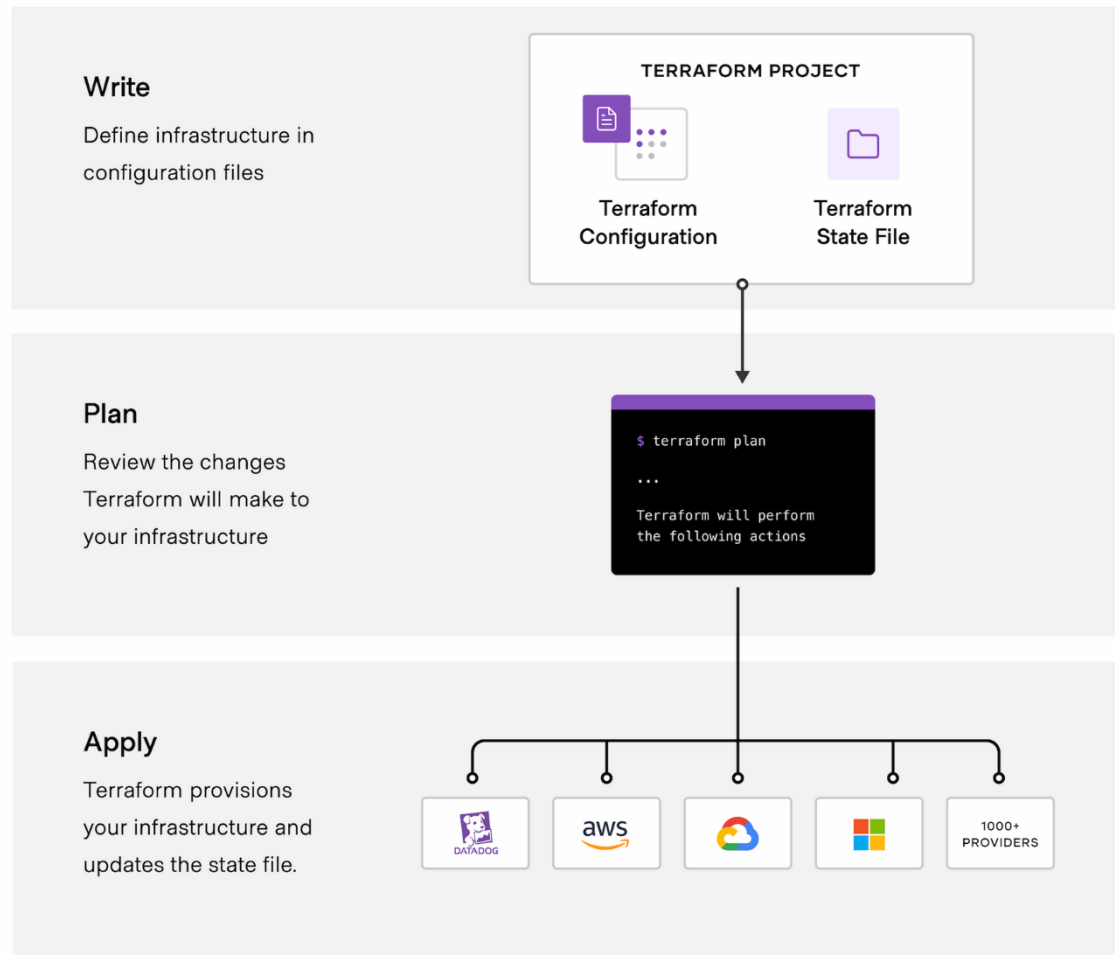


그림 3. Terraform 의 Write-Plan-Apply 작업 절차 [30]

그림 3 은 Terraform 구성을 작성하는 작성(Write), 변경 예정 내용을 확인하는 계획 검토(Plan), 승인된 변경을 반영하는 적용(Apply) 흐름을 나타낸다. Terraform 은 구성과 상태 파일을 바탕으로 현재 인프라와 목표 구성을 비교하고, 공급자 플러그인을 통해 필요한 작업을 수행한다.

이러한 선언형 방식은 여러 클라우드 리소스로 구성되는 인프라를 코드로 표현하고 반복적으로 구성하는 데 활용할 수 있다.

2.4.3. OpenTofu

OpenTofu 는 인프라 구성을 코드로 작성하고 관리할 수 있도록 지원하는 Infrastructure

as Code(IaC) 도구이며, 작성된 IaC 코드의 정적 검증에도 활용할 수 있다.

OpenTofu의 `tofu validate` 명령은 실제 클라우드 자원이나 원격 State에 접근하지 않고 작성된 구성 파일을 분석하여 문법적 오류와 내부 일관성을 검사한다. 이를 통해 리소스와 모듈에 사용된 속성 및 인자의 이름이 올바른지, 전달되는 값의 타입이 적절한지와 같은 오류를 실제 인프라를 생성하기 전에 확인할 수 있다. 검증을 수행하기 위해서는 필요한 Provider와 Module을 사용할 수 있도록 작업 디렉터리를 먼저 초기화해야 하며, 이후 `tofu validate`를 실행하여 구성의 유효성을 검사한다.

이러한 검증 과정은 실제 인프라를 변경하지 않고 IaC 코드 자체를 대상으로 오류를 확인할 수 있으므로 코드 작성 단계에서 정적 검사 용도로 활용할 수 있다.

2.4.4. Amazon Web Services(AWS)

Amazon Web Services(AWS)[16]는 컴퓨팅, 네트워크, 저장소, 데이터베이스, 보안 등 다양한 클라우드 서비스를 제공하는 클라우드 컴퓨팅 플랫폼이다. AWS의 주요 서비스 중 Amazon EC2는 애플리케이션을 실행할 수 있는 가상 서버를 제공하고, Amazon VPC는 AWS 환경에서 논리적으로 격리된 가상 네트워크를 구성할 수 있도록 한다. VPC 내부의 subnet은 IP 주소 범위를 나누어 자원을 배치할 네트워크 영역을 구성한다. Amazon Elastic Block Store(EBS)는 EC2 인스턴스에서 사용할 수 있는 블록 저장소를 제공하며, Elastic Load Balancing(ELB)은 들어오는 애플리케이션 트래픽을 여러 EC2 인스턴스, 컨테이너, IP 주소 등의 대상으로 분산한다. 이러한 리소스들은 컴퓨팅, 네트워크, 저장소와 트래픽 분산으로 구성되는 클라우드 네이티브 애플리케이션의 배포 환경을 구성하는 데 활용할 수 있다.

2.4.5. Microsoft Azure

Microsoft Azure[17]는 컴퓨팅, 네트워크, 저장소, 데이터베이스와 애플리케이션 실행 등 다양한 클라우드 서비스를 제공하는 클라우드 컴퓨팅 플랫폼이다. Microsoft Azure의 주요 서비스 중 Azure Virtual Machines는 애플리케이션을 실행할 수 있는 가상 서버를 제공하고, Azure Virtual Network는 Azure 환경에서 가상 네트워크를 구성할 수 있도록 한다. Virtual Network 내부의 Subnet은 IP 주소 범위를 나누어 자원을 배치할 네트워크 영역을 구성한다. Azure Managed Disks는 Virtual Machines에서 사용할 수 있는 블록 수준의 스토리지를 제공한다. Azure 역시 VM(가상 머신), 네트워크와 저장소 리소스를 제공하므로 클라우드 네이티브 애플리케이션의 배포 환경을 구성하기 위한 대상 플랫폼으로 활용할 수 있다.

2.4.6. Google Cloud Platform(GCP)

Google Cloud Platform[18]은 컴퓨팅, 네트워크, 저장소, 데이터베이스와 데이터 분석 등 다양한 클라우드 서비스를 제공하는 클라우드 컴퓨팅 플랫폼이다. 주요 서비스 중 Compute Engine 은 애플리케이션을 실행할 수 있는 VM 을 제공하고, Virtual Private Cloud(VPC)는 Google Cloud 환경의 자원들이 통신할 수 있는 가상 네트워크를 제공한다. Cloud Storage 는 데이터를 객체 형태로 저장할 수 있는 저장소를 제공하며, BigQuery 는 대규모 데이터를 저장하고 분석할 수 있는 데이터 분석 서비스를 제공한다. GCP 에서 제공하는 컴퓨팅 및 네트워크 리소스 또한 애플리케이션의 요구조건에 따른 클라우드 환경을 구성하는 데 활용할 수 있다.

3. 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 지원 방법

3.1. 기술 개요

본 연구에서 제안하는 기술은 요구사항 분석, 시스템 설계, 시스템 구현, 테스트를 담당하는 네 개의 AI 에이전트가 사용자가 자연어로 제시한 클라우드 네이티브 애플리케이션에 대한 개발 요구사항에 대한 작업을 순차적으로 진행하는 멀티 AI 에이전트 기반 개발 지원 기술이다. 다음 그림 4 는 각 에이전트의 구성과 수행하는 작업에 대해 나타낸다.

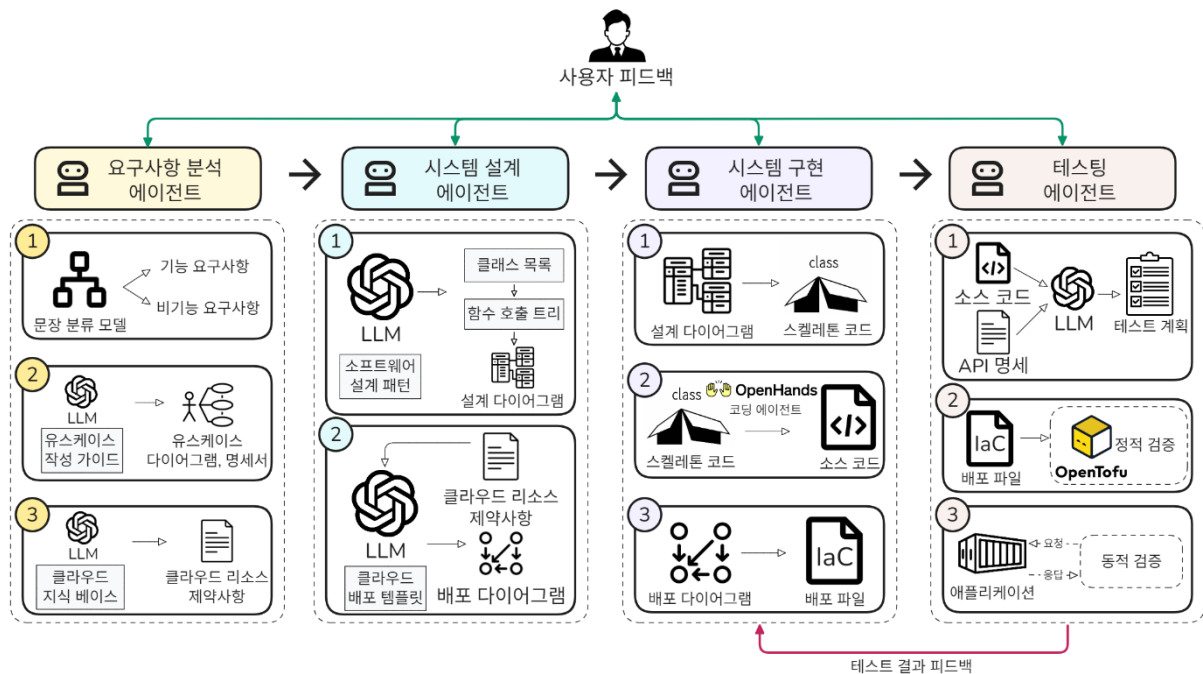


그림 4. 본 시스템의 AI 에이전트 구성 및 작업 흐름

그림 4 에 나타난 바와 같이, 제안하는 시스템은 요구사항 분석 에이전트, 시스템 설계 에이전트, 시스템 구현 에이전트, 테스트 에이전트로 구분된다. 각 에이전트는 정해진 형식의 개발 산출물을 생성하며, 각 산출물들은 사용자 피드백 과정을 거친 이후 다음 단계 에이전트의 입력으로 사용된다.

요구사항 분석 에이전트는 사용자가 입력한 요구사항으로부터 기능 요구사항 및 비기능 요구사항을 분류하고, 액터 및 유스케이스를 식별하여 유스케이스 명세서 및 다이어그램을 생성한다. 또한 클라우드 환경 구축에 필요한 리소스 별 제약사항을 지식 베이스(Knowledge base)을 기반으로 도출한다.

시스템 설계 에이전트는 요구사항 명세서를 바탕으로 애플리케이션의 설계도가 되는 클래스 다이어그램, 시퀀스 다이어그램, ERD, 그리고 API 명세서를 생성하며, 애플리케이션이 배포되는 클라우드 환경 및 리소스 구성 방식을 나타내는 배포 다이어그램을 생성한다.

시스템 구현 에이전트는 애플리케이션 설계에 정의된 범위와 기준에 따라 소스코드, 설정 파일, 실행 환경 및 필요한 산출물을 생성한다. 단위 테스트 및 통합 테스트를 통해 소스코드를 검증하며, 테스트 가능한 애플리케이션과 그 배포 구성 설정인 IaC 코드 및 배포 스크립트를 산출한다.

테스팅 에이전트는 구현 결과물을 요구사항 명세서와 시스템 설계의 기준에 따라 정적 및 동적 검증을 수행한다. 정적 검증을 통해 배포 구성에 이상이 없는지 확인하며, 동적 검증을 통해 기능이 정상적으로 동작하는지, 여러 기능이 함께 동작할 때 문제가 없는지를 확인한다. 테스트 결과는 통과 여부, 발견된 결함을 포함한 테스트 보고서로 기록된다. 테스트 과정에서 결함이 확인되면, 해당 내용을 시스템 구현 에이전트로 전달하여 수정 작업을 수행하며, 수정된 결과물은 다시 테스트 과정을 거친다.

3.2. 클라우드 리소스 분석

클라우드 리소스 분석은 클라우드 네이티브 애플리케이션의 배포 환경을 결정하기 위해 리소스 간 의존 관계, 비용 및 성능 관련 사양을 검토하는 과정이다. 본 시스템은 세 분석 결과를 바탕으로 사용자가 요구한 조건에 맞는 클라우드 리소스 구성을 제시한다. 각 분석의 내용은 다음과 같다.

- **클라우드 리소스 간 배포 의존성 분석:** 클라우드 리소스 간 배포 의존성 분석은 특정 리소스를 생성하고 사용할 때 함께 필요한 다른 리소스와 그 연결 관계를 확인하는 과정이다. 예를 들어 가상머신을 배포하려면 네트워크와 서버넷이 필요하며, 외부 요청을 받으려면 방화벽과 공인 IP 등의 네트워크 구성이 연결되어야 한다. 본 연구는 이러한 선행 조건과 참조 관계를 분석하여 애플리케이션 배포에 필요한 리소스 구성과 생성 순서를 도출한다.
- **클라우드 리소스 사용 비용 분석:** 클라우드 리소스 사용 비용 분석은 CSP 가 리소스별로 제시하는 비용 단위 및 변수를 조사하고, 이를 바탕으로 리소스 구성의 예상 사용 비용을 계산하는 과정이다. 클라우드 리소스는 실행 시간, 저장 용량, 요청 횟수 등에 따라 비용이 달라진다. 본 연구는 이러한 비용 기준을 리소스별 계산식으로 정리하여 사용자가 제시한 비용 제약사항에

부합하는 후보 리소스 구성을 선택할 수 있도록 한다.

- **클라우드 리소스 성능 분석:** 클라우드 리소스 성능 분석은 사용자가 요구할 최소 컴퓨팅 용량을 바탕으로 사용할 수 있는 리소스 후보를 찾을 수 있도록 리소스의 성능을 조사하는 과정이다. 본 연구에서는 클라우드 리소스 명세에 정의되어 있는 VM 의 가상 CPU 수와 메모리 용량을 주요 비교 기준으로 사용하여, 사용자의 최소 요구량을 충족하지 못하는 후보는 제외하고, 조건을 만족하는 후보의 사양을 제시하여 사용자가 배포할 리소스를 선택할 수 있도록 한다.

본 시스템은 클라우드 리소스 분석을 바탕으로 사용자가 제공하는 제약사항과 애플리케이션 특성을 고려하여 클라우드 리소스 선택 및 배포를 포함한 클라우드 환경 가이드라인을 제공한다. 다음 그림 5 는 이러한 가이드라인 생성 과정에 대해 표현한 것이다.

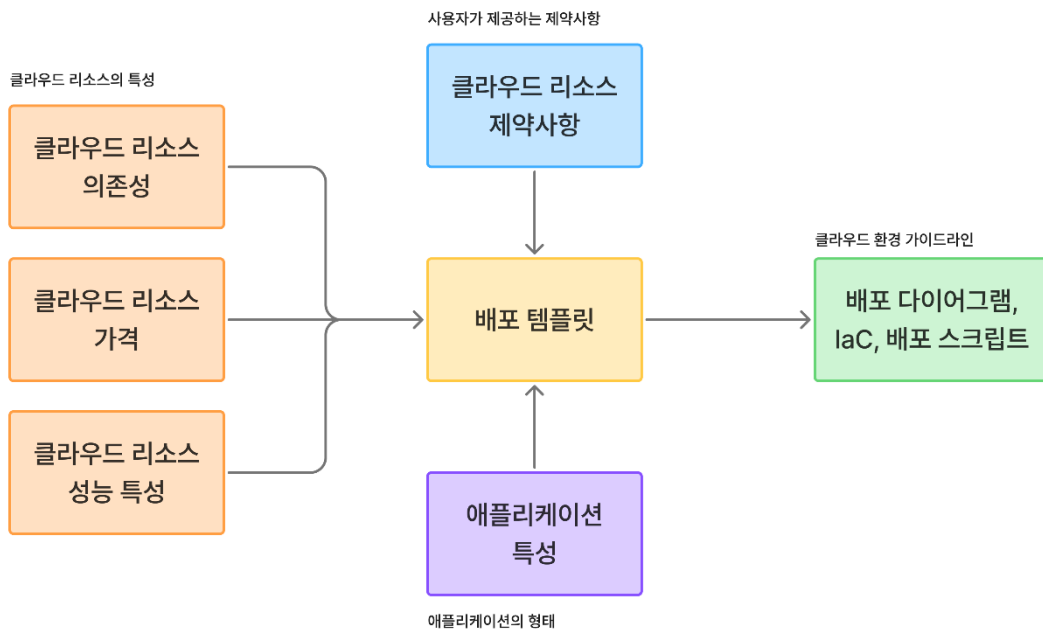


그림 5. 클라우드 리소스 분석을 통한 클라우드 환경 가이드라인 생성 과정

그림 5 에서 나타난 클라우드 환경 가이드라인은 클라우드 리소스 분석을 통해 도출한 클라우드 리소스 특성을 기반으로, 사용자의 클라우드 리소스 제약사항과 애플리케이션 특성을 반영하여 도출된다.

클라우드 리소스 간 배포 의존성 분석은 클라우드 리소스 간 배포 선후 관계를 기반으로 클라우드 리소스 간 의존성을 도출하는 과정이다. 다음 표 1 은 각 CSP 별 클라우드 리소스 유형에 따른 명칭을 조사한 결과이다.

표 1. CSP 별로 제공되는 클라우드 리소스의 유형과 명칭

카테고리	리소스 유형	AWS	Azure	GCP
컴퓨팅	가상 머신 (VM)	Amazon EC2 Instance	Azure Virtual Machine	Google Cloud Compute Engine Instance
	자동 확장형 VM	AWS Auto Scaling Group (ASG)	Azure Virtual Machine Scale Sets (VMSS)	Google Cloud Managed Instance Group (MIG)
네트워크	가상 네트워크	Amazon VPC	Azure Virtual Network (VNet)	Google Cloud VPC Network
	서브넷	Amazon VPC Subnet	Azure Subnet	Google Cloud Subnetwork
	공인 IP 주소	Amazon Elastic IP Address (EIP)	Azure Public IP Address	Google Cloud External IP Address
	로드밸런서	AWS Network Load Balancer (NLB)	Azure Load Balancer (Standard)	Regional External Passthrough Network Load Balancer
보안	방화벽	AWS Security Group	Azure Network Security Group (NSG)	Google Cloud Firewall Rule
	암호 저장소 및 권한	AWS Secrets Manager / IAM Policy	Azure Key Vault / Key Vault Secrets User	Google Cloud Secret Manager / IAM Policy Binding
스토리지	블록 스토리지	Amazon EBS Volume	Azure Managed Disk	Google Cloud Persistent Disk
컨테이너	컨테이너 레지스트리	Amazon ECR	Azure Container Registry (ACR)	Google Cloud Artifact Registry Repository

표 1 에 나타난 바와 같이 CSP 별로 같은 기능을 수행하는 클라우드 리소스가 서로 다르게 제공됨을 알 수 있다. 또한 각 리소스는 다른 리소스에 의존하여 생성되어야 하는 경우가 존재한다. 다음 그림 6 은 대표적인 예시로, AWS 에서 VM 생성 기능에 필요한 리소스들의 의존성 분석 결과를 나타낸다.

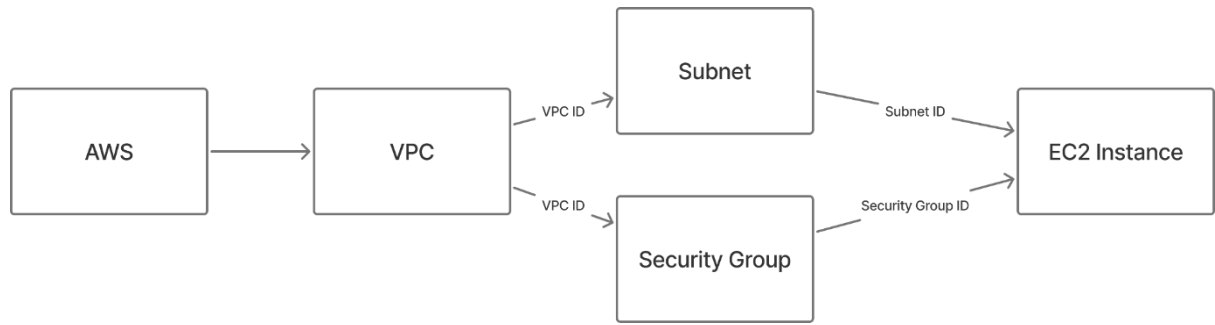


그림 6. AWS 기준 VM 생성 의존성 분석 결과

그림 6 에서 나타난 의존성 분석 결과에 따르면, AWS 에서 VM 역할을 하는 EC2 인스턴스를 생성하기 위해서는 Subnet ID 가 요구되며, Subnet 을 생성하기 위해서는 VPC ID 가 요구된다. 또한, EC2 인스턴스의 ENI(Elastic Network Interface, 네트워크 카드)는 기본적으로 인/아웃바운드 트래픽에 대해 프로토콜/포트를 제한하는 역할을 수행하는 Security Group 에 연결된다. 이러한 의존성을 고려하면, EC2 인스턴스를 생성하기 위해서는 먼저 VPC, Subnet, Security Group 이라는 리소스가 선행되어야 함을 알 수 있다.

클라우드 리소스 사용 비용 분석은 CSP 별로 제공되는 클라우드 리소스의 가격 정책을 취합하고, 이를 기반으로 리소스 별 사용 비용 계산식을 도출하는 과정이다. 다음 표 2 는 CSP 별로 제공하는 리소스의 사용 비용에 대한 계산식을 나타낸다.

표 2. 각 CSP 별 클라우드 리소스 사용 비용 계산식

CSP	리소스	계산식
AWS	Amazon EC2 Instance	pricePerUnit.USD * 인스턴스 수 * 시간
	Amazon Elastic IP Address	pricePerUnit.USD * 주소 수 * 시간
Compute (컴퓨팅)	AWS NAT Gateway	pricePerUnit.USD(시간 단위) * NAT 수 * 시간 pricePerUnit.USD(데이터 단위) * 처리한 데이터량
Compute (컴퓨팅)	AWS VPC Endpoint	pricePerUnit.USD(시간 단위) * 엔드포인트 수 * 시간 pricePerUnit.USD(데이터 단위) * 처리한 데이터량
	AWS Secrets Manager	pricePerUnit.USD(암호 개수 단위) * 암호 수 pricePerUnit.USD(암호 호출 단위) * 호출 횟수
	Amazon EBS Volume	pricePerUnit.USD(용량 단위) * 용량 * 시간 pricePerUnit.USD(IOPS 단위) * IOPS
	Amazon ECR	pricePerUnit.USD(용량 단위) * 저장 용량 * 시간 pricePerUnit.USD(용량 단위) * 전송 용량
	AWS Network Load Balancer	pricePerUnit.USD(시간 단위) * NLB 수 * 시간

		pricePerUnit.USD(트래픽 단위) * 시간당 처리한 트래픽 평균 * 시간
Azure	Azure Virtual Machine	retailPrice * 인스턴스 수 * 시간
	Azure Public IP Address	retailPrice * 주소 수 * 시간
	Azure NAT Gateway	retailPrice(시간 단위) * NAT 수 * 시간 retailPrice(데이터 단위) * 처리한 데이터량
	Azure Private Endpoint	retailPrice(시간 단위) * 엔드포인트 수 * 시간 retailPrice(데이터 단위) * 인바운드 데이터량 retailPrice(데이터 단위) * 아웃바운드 데이터량
	Azure Key Vault	retailPrice(암호 작업 단위) * 작업 횟수
	Azure Managed Disk	retailPrice(시간 단위) * 시간
	Azure Container Registry	retailPrice(시간 단위) * 시간 retailPrice(용량 단위) * 전송 용량
	Azure Load Balancer	retailPrice(규칙 개수 단위) * 규칙 개수 retailPrice(트래픽 단위) * 처리한 트래픽 * 시간
GCP	Google Cloud Compute Engine Instance	unitPrice(vCPU 단위) * vCPU 수 * 인스턴스 수 * 시간 unitPrice(RAM 용량 단위) * RAM 용량 * 인스턴스 수 * 시간
	Google Cloud External IP Address	unitPrice(개수 단위) * 사용중인 수 * 시간
	Google Cloud Cloud NAT	unitPrice(처리 VM 수 단위) * VM 수 * 시간 unitPrice(데이터 단위) * 처리한 데이터 unitPrice(IP 개수 단위) * IP 개수 unitPrice(IP 개수 단위) * IP 개수
	Google Cloud Secret Manager	unitPrice(암호 개수 단위) * 암호 수
	Google Cloud Persistent Disk	unitPrice(용량 단위) * 용량 * 시간
	Google Cloud Artifact Registry Repository	unitPrice(용량 단위) * 저장 용량 * 시간
	Regional External Passthrough Network Load Balancer	unitPrice(개수 단위) * 해당 리전의 포워딩 규칙 개수 * 시간 unitPrice(데이터 단위) * 인바운드 데이터량 unitPrice(데이터 단위) * 아웃바운드 데이터량

표 2 에 나타난 바와 같이 각 리소스들은 다양한 사용 단위들을 가지며, 리소스가 복수의 사용 단위를 가지는 경우 해당 단위로 계산된 가격들의 합을 비용으로 요구한다는 것을 알 수 있다.

클라우드 리소스 성능 분석은 최소 가상 CPU 및 메모리로 구성되는 성능 제약사항을

만족하는 리소스 풀을 사용자에게 제공하기 위해, 각 VM 들의 가상 CPU 및 메모리를 조사한 결과를 정리하여 제약사항 기반의 필터에 적용하는 과정이다. 먼저 요구사항 분석 단계에서 CSP, 리전, 최소 vCPU 와 최소 메모리 값을 추출하여 클라우드 리소스 명세에 저장한다. 이후 선택된 CSP 와 리전의 VM 목록에서 VM 의 vCPU(CPU 칩셋의 개수) 및 메모리가 사용자가 요구한 vCPU 및 메모리 용량을 모두 만족하는 후보를 검색한다. 검색된 후보군은 비용을 기준으로 정렬되어 사용자에게 각 후보별 세부 정보와 함께 제공된다. 사용자가 선택한 VM 및 개수는 배포 다이어그램 및 배포 파일에 반영된다. 따라서 제안하는 시스템은 사용자가 성능 제약사항, 비용 및 세부 정보를 함께 고려하여 VM 을 선택할 수 있도록 지원한다.

3.3. 요구사항 분석 에이전트

요구사항 분석 에이전트는 자연어로 입력된 클라우드 네이티브 애플리케이션 요구사항을 분석하여 유스케이스 명세 및 다이어그램을 생성하는 역할을 수행한다. 에이전트의 작업 과정은 사용자 요구사항 구체화 및 분류(기능적/비기능적 요구사항), 액터 및 유스케이스 식별, 유스케이스 명세 작성, 유스케이스 다이어그램 생성의 네 단계의 작업과 클라우드 리소스 명세를 생성하는 작업으로 구성된다. 그림 7 은 요구사항 분석 에이전트의 작업 순서를 나타낸다.



그림 7. 요구사항 분석 에이전트의 작업 순서

요구사항 정제 단계에서는 사용자가 입력한 초기 요구사항을 설계와 구현에 활용할 수 있는 구체적인 형태로 정리한다. 초기 요구사항에는 기능이나 조건이 명확하지 않은 표현이 포함될 수 있으며, 이러한 모호함은 이후 설계와 구현 과정에서 요구사항이 충분히 반영되지 않는 원인이 될 수 있다. 본 연구에서는 LLM 에 요구사항 예시를 함께 제공하는 Few-shot 프롬프팅을 적용하여 정제 과정을 구현하였다. 예시는 기존 소프트웨어 요구사항 문서로 구성된 PURE 데이터셋에서 추출하였으며, 이를 통해 정제된 요구사항의 표현 방식과 구체화 수준을 안내하였다.

또한 액터 및 유스케이스는 기능적 요구사항으로부터 도출할 수 있으므로, 구체화된

요구사항으로부터 기능적 요구사항을 분류할 방법이 필요하기에 자연어 문장 분류에 널리 쓰이는 BERT 모델을 파인 튜닝하여 분류기로 사용하였다. 파인 튜닝을 위한 학습 데이터는 앞서 언급한 PURE 데이터셋을 포함한 여러 요구사항 문서들에서 추출된 요구사항 문장들을 기능/비기능으로 태깅한 데이터셋(FR_NFR_Dataset)을 활용하였다.

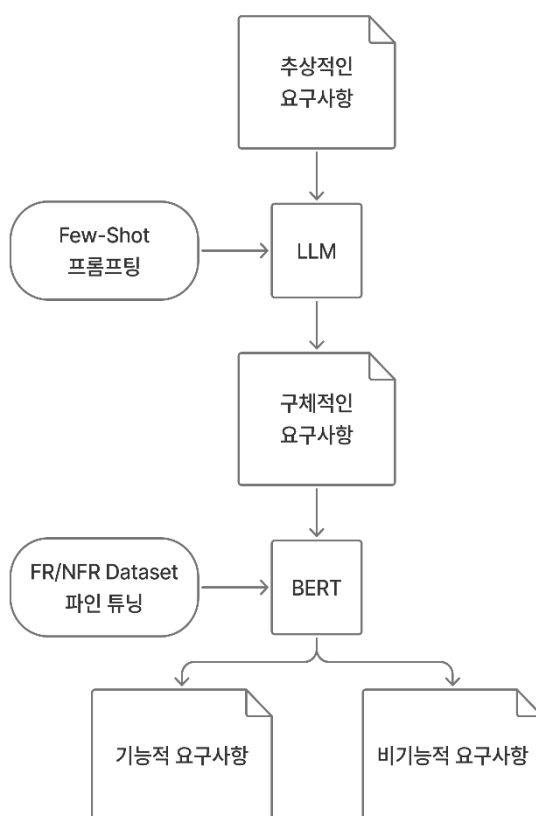


그림 8. 요구사항 정제 과정

그림 8에서는 이러한 요구사항 정제 과정을 나타낸다. 먼저 Few-shot 예시를 포함한 프롬프트를 받은 LLM이 사용자가 입력한 요구사항을 구체화하고, 구체화된 요구사항은 FR_NFR_Dataset으로 파인 튜닝한 BERT 분류기에 입력되며, 분류기는 각 문장을 기능적 요구사항과 비기능적 요구사항으로 분류함으로써 요구사항 정제가 완료된다.

유스케이스 도출 단계에서는 정제된 요구사항을 바탕으로 시스템과 상호작용하는 액터와 각 액터가 달성하려는 목표를 식별하고, 이를 유스케이스로 구성한다. 각 유스케이스에는 관련 기능·비기능적 요구사항의 ID를 연결하여 요구사항과의 대응 관계를 기록하고, 모든 기능적 요구사항이 하나 이상의 유스케이스에 반영되었는지 확인한다. 액터와 유스케이스 도출 및 유스케이스별 명세 생성에는 LLM을 활용하며, 일관된 작성

기준을 제공하기 위해 유스케이스 작성법을 기술한 도서[19]를 참고하여 프롬프트를 설계하였다. 또한 근거 없는 내용의 생성과 요구사항 누락을 줄이기 위해, 각 단계의 결과에 규칙 기반 검증과 LLM 기반의 의미 검증을 적용하였다. 그림 9 는 유스케이스 명세 생성 과정을 나타낸다.

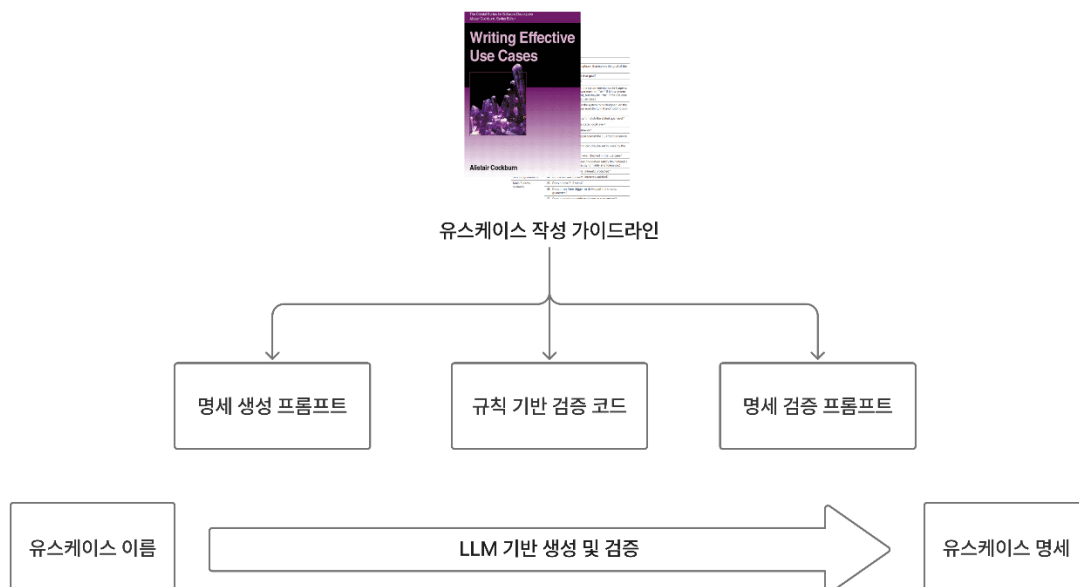


그림 9. 유스케이스 명세 생성 및 검증 과정

그림 9 에 따른 유스케이스 명세 생성이 끝나면, 액터와 유스케이스의 연관 관계, 유스케이스 사이의 include-extend 관계와 액터의 일반화 관계를 식별한다. 요구사항 분석 에이전트는 관계에 사용된 액터와 유스케이스가 실제 산출물에 존재하는지 확인하고, 모든 기능 요구사항이 적어도 하나의 유스케이스에 연결되었는지 검사한다.

3.4. 설계 에이전트

시스템 설계 에이전트는 생성된 요구사항 및 유스케이스 명세를 바탕으로 클래스 다이어그램, 시퀀스 다이어그램, API 명세, ER 다이어그램, 배포 다이어그램을 생성한다. 그림 10 은 시스템 설계 에이전트의 전체 산출물 생성 순서를 나타낸다.

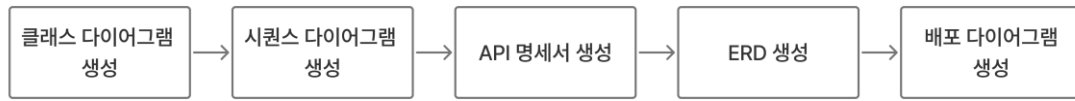


그림 10. 시스템 설계 에이전트의 전체 산출물 생성 순서

클래스 다이어그램 생성 단계에서는 요구사항 시나리오를 바탕으로 사용자와 시스템의 상호작용, 기능 처리, 지속적으로 관리할 데이터를 구분하고, 이를 각각 Boundary·Control·Entity 클래스로 구성한다. 이를 위해 LLM 프롬프트에 BCE 패턴의 역할 구분과 클래스 도출 기준을 제시하였다. Boundary 는 사용자와 시스템과의 상호작용을 기준으로 구성하고, Control 은 유스케이스 실행을 위한 로직을 기준으로 분리하며, Entity 는 유스케이스 실행과 독립적으로 관리되는 데이터를 대상으로 도출한다.

시퀀스 다이어그램 생성 단계에서는 앞서 생성된 클래스 모델의 관계와 동작 정보를 정해진 변환 규칙에 따라 시퀀스 다이어그램으로 변환한다. 이는 LLM 이 시퀀스 다이어그램을 직접 생성할 때 동일한 입력에서도 호출 주체와 순서가 달라질 수 있는 문제를 줄이고, 산출물 간 일관성과 생성 과정의 재현성을 확보하기 위한 설계이다. 변환 과정에서는 호출 간 관계에 따라 순서를 구성하고, 각 호출에 대응하는 반환 메시지와 유스케이스의 포함·확장 관계를 반영한다. 이후 호출과 반환의 일치함, 액터 및 메서드 참조의 유효성, 유스케이스의 반영 여부를 검증한다.

API 명세 생성에서는 LLM 이 Boundary 에서 Control 로 이어지는 상호 작용을 기준으로, OpenAPI 명세 형태를 기반으로 생성한다. 예를 들어 유스케이스와 시퀀스 다이어그램으로부터 사용자와 상호작용하는 동작을 식별하고, 이에 대응하는 Boundary-Control 호출을 찾아 요청 데이터와 Control 인자의 매핑을 먼저 정의한다. 이후 기본 흐름과 정상·예외 흐름을 바탕으로 HTTP 메서드와 응답을 정의하고, 클래스 다이어그램의 클래스를 반영하여 구조화된 API 모델을 도출한다. 시스템은 이 모델을 OpenAPI 3.1 명세로 변환한다.

ERD 생성은 클래스 단계에서 Entity 클래스와 데이터 클래스를 기준으로 생성한다. 예를 들어 Entity 클래스의 속성과 자료형을 테이블의 컬럼과 데이터 타입으로 변환하고, 식별자 정보를 바탕으로 기본키를 구성한다. 이후 클래스 간 관계와 다중도에 따라 외래키 또는 연결 테이블을 생성하여 논리 데이터 모델을 구성하고, 이를 ERD 로 표현한다.

배포 다이어그램 생성 단계에서는 클라우드 리소스 요구사항에 기반하여 생성된 리소스 가이드라인을 기준으로 생성된다. 설계 산출물 및 클라우드 리소스 요구사항을 기반으로 사전 구성된 배포 다이어그램 템플릿을 활용해 클라우드 리소스 생성 관계 및 런타임 관계를 보여주는 배포 다이어그램을 생성한다.

3.5. 구현 에이전트

구현 에이전트는 시스템 설계 에이전트의 산출물을 기반으로 실제 애플리케이션을 수행하기 위한 서비스 동작 코드와, 클라우드 네이티브 환경을 구축하기 위한 클라우드 리소스 배포 파일을 생성한다.

구현 에이전트는 시스템 설계 에이전트의 산출물인 클래스 다이어그램, 시퀀스 다이어그램, API 명세, ER 다이어그램, 배포 다이어그램에 맞추어 생성된다. 클래스 다이어그램과 시퀀스 다이어그램을 기준으로 생성한 템플릿 코드(스켈레톤 코드)를 생성하고, 요구사항 분석 에이전트에서 도출되었던 유스케이스 명세와 API 명세에 맞추어 스켈레톤 코드 내부의 시스템 동작을 실현하는 코드를 생성한다. 또한 ER 다이어그램과 클래스 다이어그램을 기준으로 데이터베이스 테이블과 스키마를 구성하는 SQL 문을 작성한다. 배포 다이어그램을 기준으로 리소스 구성 파일(리소스의 구성을 명세하는 파일) 및 리소스 배포 파일(리소스 배포에 필요한 정보를 담은 파일)을 생성한다. 이 과정을 통해 생성된 파일들을 바탕으로 배포를 수행하여, 서비스를 동작 시키기 위한 애플리케이션을 구축한다.

3.6. 테스트 에이전트

테스팅 에이전트는 구현 에이전트가 생성한 애플리케이션과 배포 관련 산출물을 입력으로 받아, 요구사항과 설계 내용에 맞게 구성되었는지를 검사한다. 구현 산출물은 구조와 설정을 실행하지 않고 확인하는 정적 검증과, 별도의 환경에서 실제로 실행하여 동작을 확인하는 동적 검증을 거친다. 그림 11 은 테스트 에이전트의 전체 검증 흐름을 나타낸다.

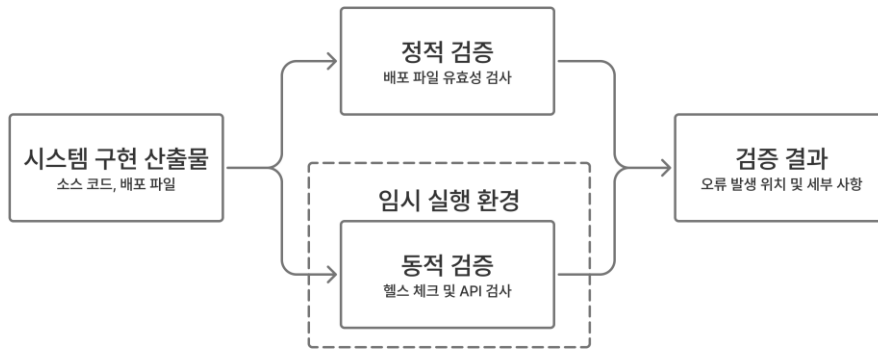


그림 11. 테스트 에이전트의 정적·동적 검증 흐름

그림 11 과 같이 테스트 에이전트는 검증할 구현 산출물의 버전을 고정한 뒤 정적 검증과 동적 검증을 수행한다.

정적 검증은 생성된 애플리케이션을 실행하지 않은 상태에서 구현 산출물이 배포에 필요한 구조와 설정을 갖추었는지 확인하는 과정이다. 테스트 에이전트는 구현 산출물이 배포에 필요한 파일과 설정을 갖추었는지 확인하고, 설정과 클라우드 리소스 간 연결이 올바른 지 검사한다. 또한 자동화된 정적 분석을 통해 암호 노출, 보안상 잘못된 설정, 배포 스크립트와 인프라 코드의 문법 및 구성 오류를 확인한다. 발견된 문제는 해당 위치와 원인을 함께 기록하여 구현 산출물의 수정에 활용한다.

동적 검증은 정적 검증과 동일한 구현 산출물을 컨테이너 기반의 격리된 시험 환경에서 실행하여 애플리케이션의 실제 동작을 검증하는 과정이다. 먼저 서비스가 요청을 처리할 준비가 되었는지 확인한다. 이후 테스트 에이전트의 LLM 은 요구사항, 유스케이스 및 API 명세를 바탕으로 API 호출 순서와 성공 조건을 포함한 시험 시나리오를 구성한다. 시스템은 해당 시나리오에 따라 API 를 호출하고, 응답 상태와 데이터 구조가 명세에 부합하는지, 호출 결과가 요구된 기능을 충족하는지 검사한다. 발견된 문제는 발생 위치와 원인을 함께 기록하여 구현 산출물의 수정에 활용한다.

4. 사례 애플리케이션 생성 및 AWS 배포 사례

본 장에서는 수강신청 애플리케이션 사례를 기준으로 입력 요구사항이 설계와 구현 산출물로 변환되는 과정과, 생성된 배포 스크립트를 사용해 AWS 에서 실행 결과를 확인하는 과정을 보인다.

4.1. 요구사항 분석 에이전트

입력은 자연어 소프트웨어 요구사항과 클라우드 자원 제약(공급자·리전·예산 등)으로 구성된다. 표 3 과 표 4 는 사례 시스템에 입력한 요구사항 및 클라우드 제약의 원문과 한국어 번역 예시를 나타낸다.

표 3. 시스템에 입력한 수강 신청 애플리케이션 자연어 요구사항 예시

번호	실제 입력 원문	한국어 번역
1	Students, professors, and academic administrators are specialized roles of a university user. An authenticated identity and role are preconditions for protected operations.	학생·교수·학사 관리자는 대학 사용자의 세부 역할이며, 인증과 권한 확인이 필요한 기능은 사용자 인증을 마치고 해당 역할을 부여받은 경우에만 사용할 수 있다.
2	A university user shall search published course offerings and inspect their course, section, instructor, meeting schedule, capacity, and remaining-seat information.	대학 사용자는 개설 강좌를 검색하고 교과목·분반·담당 교수·시간·정원·잔여 좌석을 확인할 수 있어야 한다.
3	A student shall register for a selected course offering during its registration period and receive the result.	학생은 수강신청 기간에 선택한 강좌를 신청하고 처리 결과를 받아야 한다.

표 4. 시스템에 입력한 수강 신청 애플리케이션의 클라우드 제약사항

구분	내용
실제 입력 원문	This application will pilot the service on AWS in the Seoul Region, with a monthly cloud budget not exceeding USD 5. The workload must provide at least 1 vCPU, 1 GiB of memory, and 8 GiB of persistent block storage, and it must run as a single virtual-machine replica. The expected traffic pattern is steady rather than spiky. The fixed research scope is Docker on Linux virtual machines provisioned with Terraform and exposed through temporary HTTP endpoints. HTTPS, TLS certificates, domain management, managed database services, Kubernetes, and serverless compute are outside the scope of this pilot.

구분	내용
한국어 번역	본 애플리케이션은 AWS 서울 리전에서 서비스를 시범 운영하며, 월간 클라우드 예산은 5 달러를 초과할 수 없다. 워크로드에는 최소 1 개의 vCPU, 1GiB 의 메모리, 8GiB 의 영구 블록 스토리지가 필요하며, 단일 가상 머신 복제본으로 실행해야 한다. 예상 트래픽 패턴은 급증형이 아닌 일정형이다. 연구 범위는 Terraform 으로 프로비저닝한 Linux 가상 머신에서 Docker 를 실행하고 임시 HTTP 엔드포인트를 제공하는 것으로 한정한다. HTTPS, TLS 인증서, 도메인 관리, 관리형 데이터베이스 서비스, Kubernetes 및 서버리스 컴퓨팅은 이번 시범 운영 범위에서 제외한다.

표 3 에는 전체 요구사항 16 개 가운데, 이후 설계 결과를 설명하는 데 사용하는 세 항목을 제시하였다. 요구사항 1 은 학생·교수·학사 관리자를 대학 사용자의 세부 역할로 정의하고, 인증이 필요한 기능은 해당 역할을 부여 받은 사용자만 이용하도록 규정한다. 요구사항 2 는 개설 강좌 검색과 상세 정보 조회를, 요구사항 3 은 신청 기간 내 수강 신청과 처리 결과 확인을 규정한다.

표 4 의 클라우드 제약사항은 배포 대상을 AWS 서울 리전의 Linux 가상머신과 Docker 로 제한하며, 월 예산과 제외 기술을 함께 전달한다.

사례 시스템의 설계 산출물을 도출하기 위해 요구사항 분석 에이전트가 입력 문장을 정제하고 기능 요구사항과 비기능 요구사항으로 분류하였다. 표 5 는 전체 분류 결과 가운데 대표 세 항목을 나타낸다.

표 5. 정제 및 분류된 요구사항 예시

번호	분류	시스템이 정제한 요구사항
요구사항 1	비기능	학생·교수·학사 관리자는 대학 사용자의 세부 역할이며, 인증과 권한 확인이 필요한 기능은 사용자 인증을 마치고 해당 역할을 부여 받은 경우에만 사용할 수 있다.
요구사항 2	기능	대학 사용자는 개설 강좌를 검색하고 교과목·분반·담당 교수·시간·정원·잔여 좌석을 확인할 수 있어야 한다.
요구사항 3	기능	학생은 수강신청 기간에 선택한 강좌를 신청하고 처리 결과를 받아야 한다.

표 5 의 요구사항 1 은 인증과 역할 확인이라는 시스템 운영 조건을 규정하므로 비기능 요구사항으로 분류되었다. 강좌 검색과 수강 신청 동작을 규정한 요구사항 2 와 3 은 기능 요구사항으로 분류되었다.

요구사항 분석은 인증, 권한, 영구 저장, 동시성 조건을 추출하고 이 중 인프라 구성에

반영할 조건을 구분한다. 데이터 저장 필요성은 저장 자원 설계의 근거가 되며, 인증·권한·동시성 조건은 애플리케이션 구현과 검증에도 전달되어야 한다. AWS 서울 리전과 월 5 달러 예산은 제약사항에서 구조화된다.

분류된 요구사항에서 기능을 수행하는 주체와 역할의 상속 관계를 분석하여 대학 사용자, 학생, 교수, 학사 관리자의 네 액터를 도출하였다. 대학 사용자 액터는 나머지 액터들의 일반화이며, 나머지 세 액터는 이를 구체화한다. 시스템이 각 액터를 생성한 결과 및 관련된 요구사항에 대한 내용은 표 6 과 같다.

표 6. 생성된 액터 목록

액터	상위 액터	설명	관련 입력
대학 사용자 (University User)	-	인증 후 공통 기능을 사용하는 대학 구성원	요구사항 1, 2
학생 (Student)	대학 사용자	강좌 검색, 신청·교환·취소, 시간표와 대기열 관리	요구사항 1, 3
교수 (Professor)	대학 사용자	담당 강좌와 수강생 명단 조회	요구사항 1
학사 관리자 (Academic Administrator)	대학 사용자	학기·신청 기간·개설 강좌 관리	요구사항 1

표 6 의 액터를 바탕으로 각 사용 목적을 유스케이스로 나누고 관련 요구사항을 연결하였다. 표 7 은 생성된 유스케이스 중, 이후 설계 예시로 사용하는 강좌 검색과 수강 신청을 나타낸다.

표 7. 유스케이스 예시

유스케이스 번호	유스케이스	주 액터	목표	관련 요구사항
UC1	강좌 검색	대학 사용자	공개된 강좌와 상세 정보 조회	2
UC2	수강 신청	학생	선택 강좌를 신청하고 결과 확인	3

생성된 각 유스케이스는 선행 조건, 기본 시나리오, 선택·예외 시나리오와 후행 조건을 갖는 유스케이스 명세로 확장되었다. 표 8 와 표 9 은 각각 확장된 유스케이스 명세의 단계 수, '수강 신청' 유스케이스의 생성된 명세를 나타낸다.

표 8. 유스케이스 명세 생성 결과 요약

(단위: 단계 수)

ID	명세	선행 조건	기본 흐름	대안 흐름	예외 흐름
UC1	Search Course Offerings	1	6	1	0
UC2	Register for Course Offering	2	5	2	0

표 9. 수강 신청 유스케이스 명세 상세 내용

구분	단계	상세 내용	연결된 요구사항
선행 조건	-	학생이 대학 사용자로 인증되어 있고 신청할 강좌를 선택함	
기본 흐름	1	학생이 선택한 강좌의 수강 신청을 요청함	요구사항 3
기본 흐름	2	시스템이 학생에게 수강 신청 권한이 있는지 확인함	요구사항 14
기본 흐름	3	신청 기간, 중복 신청, 정원, 시간표 충돌을 검사함	요구사항 5
기본 흐름	4	시스템이 학생의 수강 내역을 저장함	요구사항 3
기본 흐름	5	시스템이 신청 성공 결과를 학생에게 알림	요구사항 3
대안 흐름	2a	신청 권한이 없으면 허용되지 않음을 알림	권한 오류
대안 흐름	3a	자격 검사에 실패하면 거절 이유를 알림	신청 조건 오류

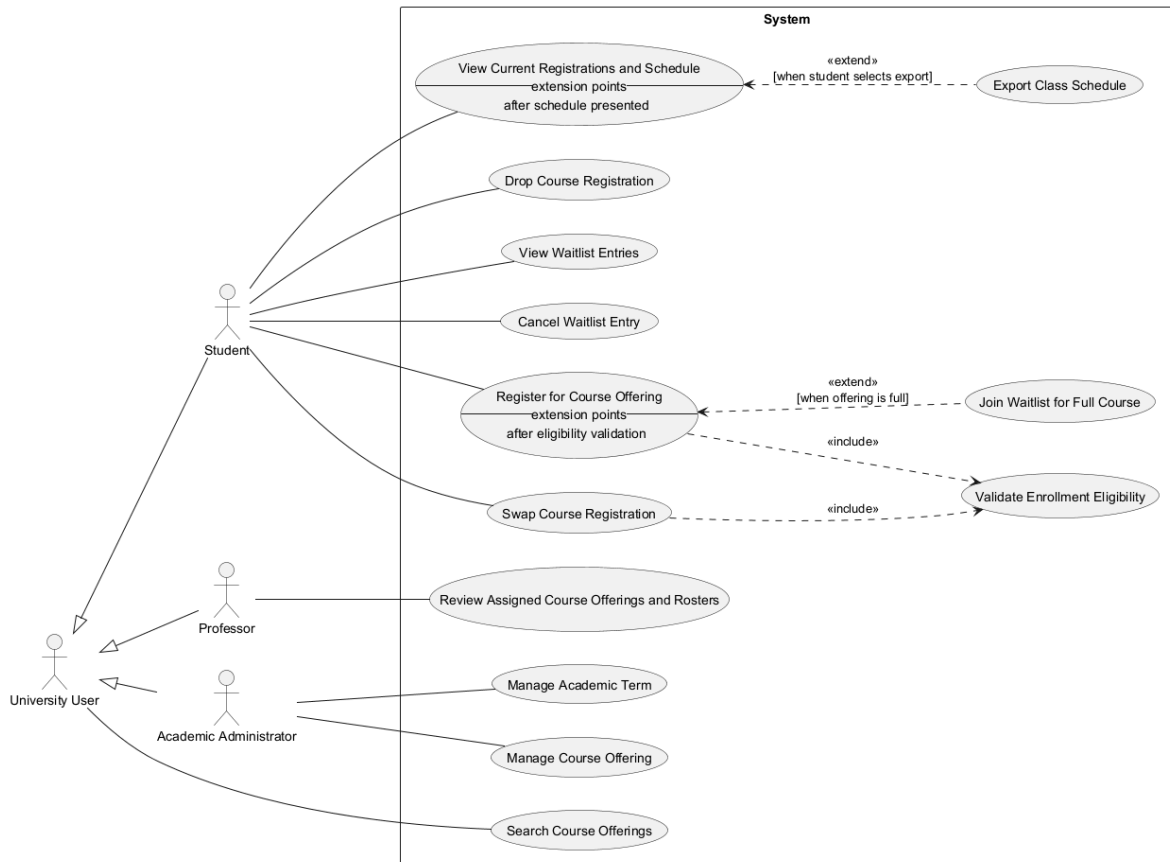


그림 12. 생성된 전체 유스케이스 다이어그램

그림 12 는 4 개 액터와 13 개 유스케이스의 관계를 보여 준다. 수강 신청과 수강 교환은 신청 자격 검사를 공통으로 사용하고, 정원이 가득 찬 경우에는 대기열 등록으로 이어진다. 시간표 내보내기는 시간표 조회 뒤에 선택할 수 있는 기능으로 표현되었다. 시스템은 해당 유스케이스 다이어그램 그림을 편집용 원본과 문서·화면 표시용 이미지로 함께 저장한다.

수강신청 애플리케이션의 초기 유스케이스 다이어그램에서는 교수의 '담당 강좌 조회'와 '수강생 명단 조회'를 유스케이스 10 하나로 표현하였다. 이는 두 개의 사용자 목표를 하나의 유스케이스로 표현하였다고 볼 수 있으며, 충분히 사용자의 피드백 사항을 적용할 수 있다고 판단하였다. 따라서, 담당 강좌 자체를 확인하는 행위와 선택한 강좌의 수강생 명단을 확인하는 행위를 서로 독립적인 사용자 목표로 표현하기 위해 유스케이스 10의 분리를 자연어 피드백으로 전달하였다. 다음 표 10은 사용자 피드백을, 그림 13은 사용자 피드백이 반영된 유스케이스 및 명세를 바탕으로 요구사항 분석 에이전트가 유스케이스 다이어그램을 다시 생성한 결과를 나타낸다.

표 10. 유스케이스 분리를 위해 자연어로 전달된 사용자 피드백

구분	내용
실제 입력 원문	Split UC10, "Review Assigned Course Offerings and Rosters," into two independent use cases. Keep UC10 for reviewing assigned course offerings, and add a new use case for viewing the student roster of a selected course offering.
한국어 번역	UC10 '담당 강좌 및 수강생 명단 조회'를 서로 독립적인 두 개의 유스케이스로 분리하라. UC10 은 담당 강좌 조회를 위한 유스케이스로 유지하고, 선택한 강좌의 수강생 명단을 조회하는 새로운 유스케이스를 추가하라.

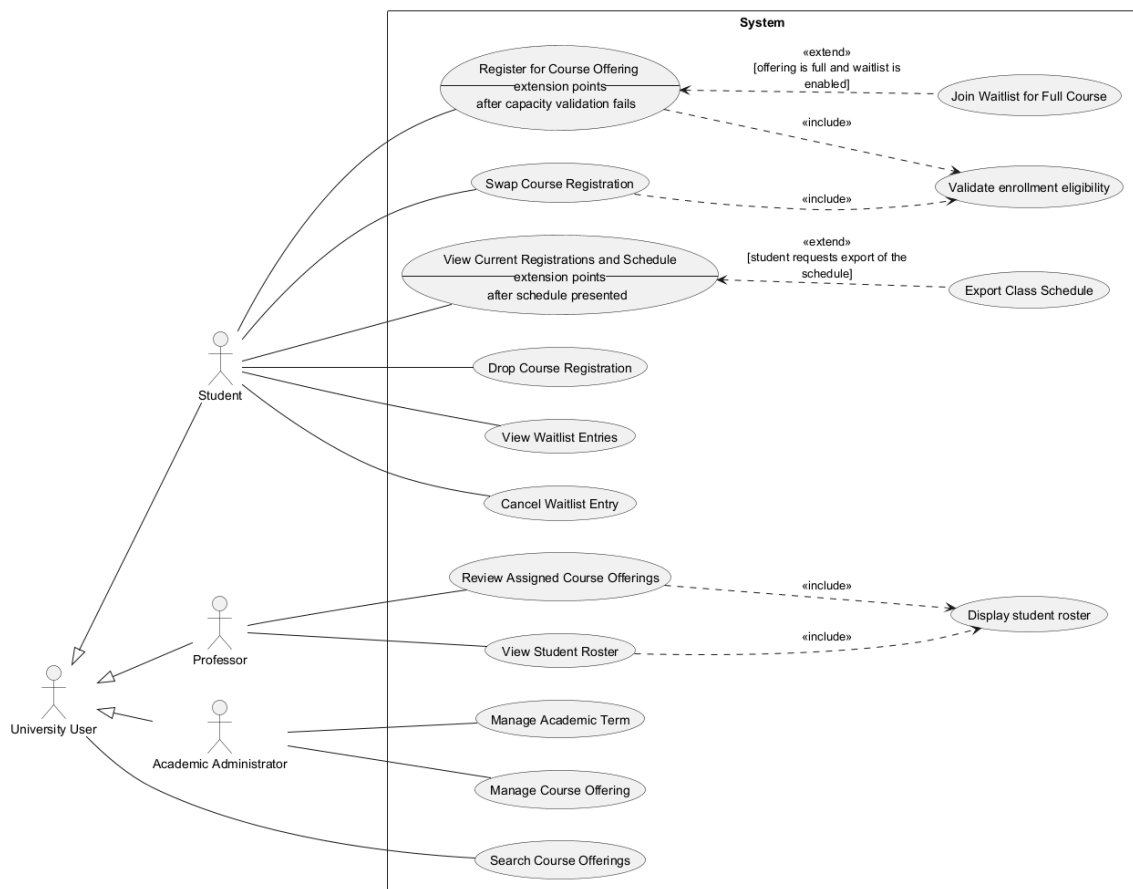


그림 13. 사용자 피드백을 적용한 후 재생성된 유스케이스 다이어그램

그림 13 에 나타난 것과 같이, 요구사항 분석 에이전트는 선택된 유스케이스 10 을 수정 대상으로 지정한 뒤 유스케이스들을 다시 생성하였다. 수정 후 유스케이스 10 은 Review Assigned Course Offerings 로 범위가 좁아졌고, View Student Roster 가 유스케이스 13 으로 추가되었다. 유스케이스 수는 13 개에서 15 개로 늘었으며, 유스케이스 10 을 제외한 기존 12 개 유스케이스는 변경되지 않았다.

4.2. 시스템 설계 에이전트

본 절에서는 시스템을 구성하는 두 번째 단계인 시스템 설계 에이전트를 통해 앞선 요구사항 분석 에이전트가 생성한 유스케이스 명세를 활용하여 시스템 설계 산출물을 도출하고, 시스템 구현 산출물 도출의 기반을 마련하는 과정을 설명한다.

클래스 다이어그램은 유스케이스 시나리오의 사용자 요청, 업무 처리 순서와 저장 대상을 각각 Boundary, Control, Entity 로 클래스를 구분하였다.

클래스 모델은 필드·연산·요청 및 반환 자료형과 관련 유스케이스를 기반으로 생성되었다. 그림 14 는 유스케이스 1, 2 명세를 기반으로 클래스 모델을 나타낸 것이다.

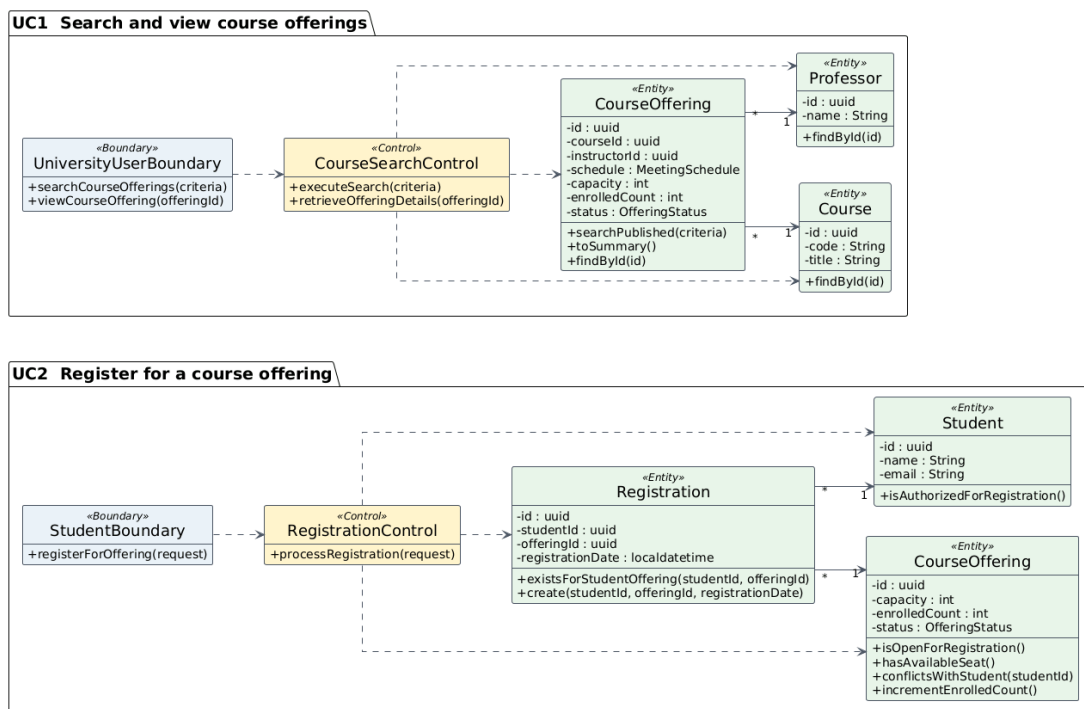


그림 14. 유스케이스 1, 2 의 클래스 다이어그램

그림 14 는 전체 클래스 모델에서 유스케이스 1 과 2 에 직접 참여하는 클래스를 나타낸 것이다.

유스케이스 1에서는 UniversityUserBoundary 클래스가 검색 조건을 받고 CourseSearchControl클래스가 CourseOffering, Course, Professor 클래스를 조회한다. 유스케이스2에서는 StudentBoundary클래스가 수강 신청을 받고 RegistrationControl 클래스가 Student와 CourseOffering클래스의 상태를 확인한 뒤 Registration 클래스를 생성한다.

Boundary는 사용자 요청과 응답, Control은 처리 순서, Entity는 조회·저장 대상에서 각각 도출되었다.

앞서 요구사항 분석 에이전트를 통해 도출한 유스케이스 명세와 시스템 설계 에이전트를 통해 도출한 클래스 다이어그램을 기반으로 컴포넌트의 호출·응답 순서를 구성하는 시퀀스 다이어그램을 도출하였다. 그림 15 와 그림 16 은 유스케이스 1, 2 의 시퀀스 다이어그램 생성 결과를 나타낸다.

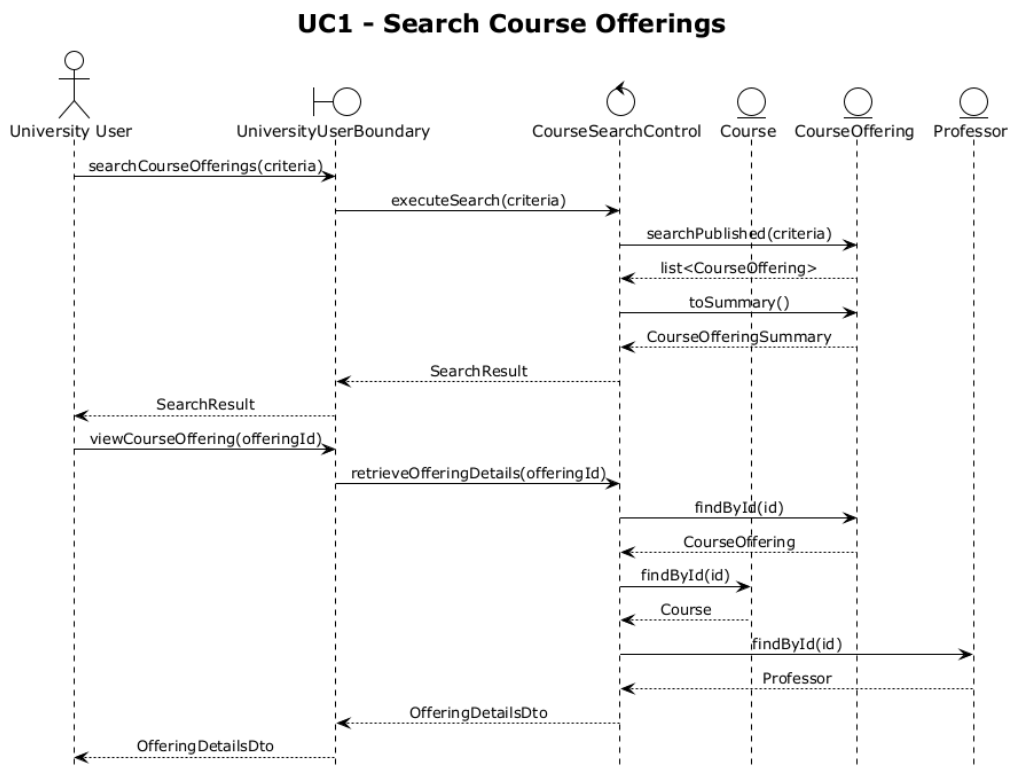


그림 15. 강좌 검색 기능의 시퀀스 다이어그램

그림 15 는 사용자의 검색 조건을 UniversityUserBoundary 클래스가 받아 CourseSearchControl 클래스에 전달하고, 개설 강좌 검색 결과와 상세 정보를 반환하는 시퀀스 다이어그램이다.

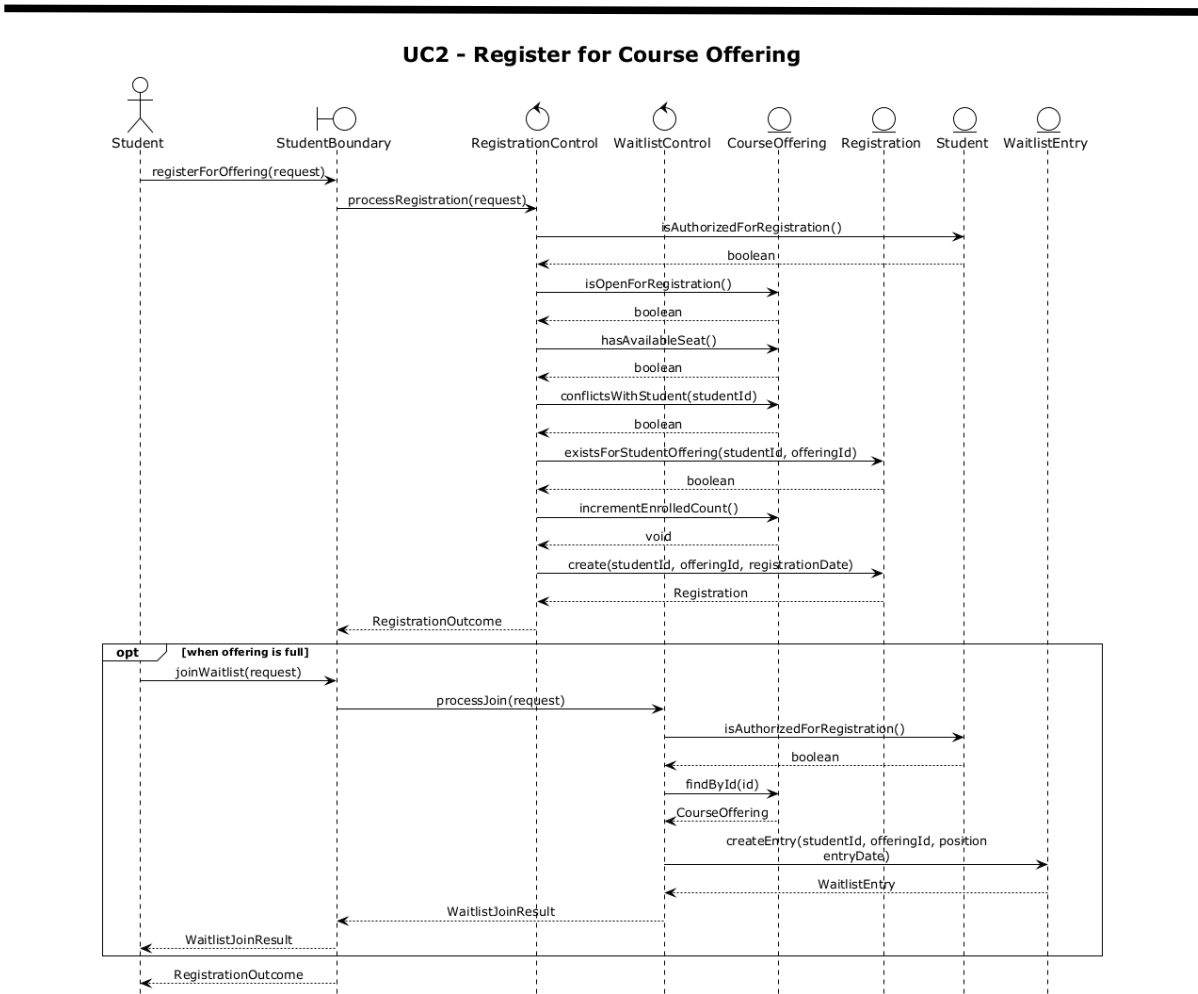


그림 16. 수강 신청 기능의 시퀀스 다이어그램

그림 16 은 StudentBoundary 클래스에서 RegistrationControl 클래스로 신청 요청이 전달되고, 권한·신청 기간·잔여 좌석·시간 충돌·중복 신청을 확인한 뒤 좌석 수와 수강 기록을 갱신하는 시퀀스 다이어그램이다. 도출된 다이어그램은 편집 가능한 PlantUML 과 문서 표시용 PNG·SVG 로 저장된다.

클래스 다이어그램의 연산과 시퀀스 다이어그램의 호출·응답 순서를 바탕으로 Open-API 3.1.0 명세를 생성하였다. 표 11 은 유스케이스 1 과 2 에서 도출된 세 API 의 경로, 요청 본문과 응답 코드를 요약한다.

표 11. 생성된 유스케이스 1, 2 의 API 명세

메서드	경로	operationId	요청	응답	UC
POST	/course-offerings/search	searchCourseOfferings	CourseSearchCriteria	200, 400, 403	UC1

메서드	경로	operationId	요청	응답	UC
GET	/course-offerings/{offeringId}	viewCourseOffering	-	200, 404, 403	UC1
POST	/registrations	registerForOffering	RegistrationRequest	201, 400, 403, 409	UC2

다음 코드는 생성된 OpenAPI JSON 에서 '수강 신청' API 중 하나인 POST /registrations 항목을 나타낸다.

```
{
  "openapi": "3.1.0",
  "info": { "title": "API", "version": "1.0.0" },
  "paths": {
    "/registrations": {
      "post": {
        "operationId": "registerForOffering",
        "x-system-control": {
          "control": "RegistrationControl",
          "method": "processRegistration",
          "arguments": { "request": "$body" },
          "outcomes": { "201": "created", "400": "validation_error", "403": "unauthorized", "409": "conflict" }
        },
        "requestBody": {
          "required": true,
          "content": { "application/json": { "schema": { "$ref": "#/components/schemas/RegistrationRequest" } } }
        },
        "responses": {
          "201": { "description": "Response", "content": { "application/json": { "schema": { "$ref": "#/components/schemas/RegistrationOutcome" } } } },
          "400": { "description": "Response", "content": { "application/json": { "schema": { "$ref": "#/components/schemas/RegistrationOutcome" } } } },
          "403": { "description": "Response", "content": { "application/json": { "schema": { "$ref": "#/components/schemas/RegistrationOutcome" } } } },
          "409": { "description": "Response", "content": { "application/json": { "schema": { "$ref": "#/components/schemas/RegistrationOutcome" } } } }
        }
      }
    }
  }
}
```

API 예시에서 operationId 는 수강 신청 작업을 registerForOffering 으로 식별하고, requestBody 는 RegistrationRequest 형식의 JSON 입력을 요구한다. responses 에는 성공 201, 입력 오류 400, 권한 오류 403, 신청 충돌 409 가 RegistrationOutcome 응답과 함께 정의되어 있다. OpenAPI 의 확장 필드인 x-system-control 은 이 API 를 RegistrationControl.processRegistration 과 연결한다. 이를 통해 '학생이 선택한 강좌를 신청한다'는 요구 사항이 수강 신청 시나리오와 처리 클래스를 거쳐 외부 API 명세로 구체화된 결과를 확인할 수 있다.

BCE 기반 클래스 모델의 Entity 를 식별자, 속성, 외래키 관계로 변환하여 ERD 를 생성

하였다. 유스케이스 1 과 2 에 필요한 저장 대상은 Course, Professor, CourseOffering, Student, Registration 의 다섯 엔티티이며, 표 12 는 해당 엔티티의 주요 필드와 관계를 나타낸다.

표 12. 유스케이스 1, 2 관련 엔티티 목록

엔티티	식별자	필드	연결 관계
Course	id	code, title, description	CourseOffering 과 1:N
Professor	id	name, email	CourseOffering 과 1:N
CourseOffering	id	courseld, instructorId, schedule, capacity, enrolledCount, status	Course-Professor 와 N:1, Registration 과 1:N
Student	id	name, email	Registration 과 1:N
Registration	id	studentId, offeringId, registrationDate	Student-CourseOffering 과 N:1

표 13 과 그림 17 은 강좌 검색과 수강 신청에 필요한 다섯 엔티티와 네 관계를 보여준다. CourseOffering 엔티티는 Course 와 Professor 엔티티를 참조하고, Registration 엔티티는 신청한 Student 엔티티와 대상 CourseOffering 엔티티를 연결한다. 다른 유스케이스에서만 사용하는 엔티티는 예시에서 제외하였다.

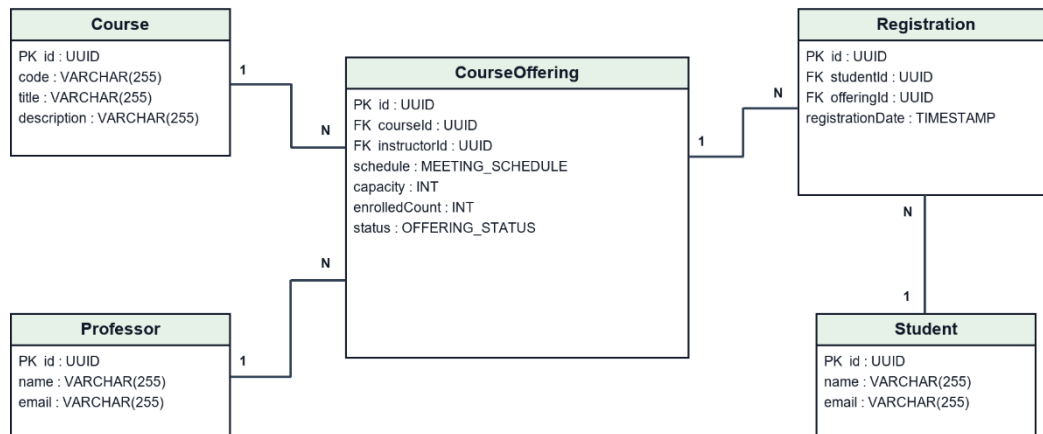


그림 17. 유스케이스 1, 2 관련 ERD

그림 17 에서 CourseOffering 엔티티의 courseld 와 instructorId 는 각각 Course 엔티티 와 Professor 엔티티를 참조한다. Registration 엔티티의 studentId 와 offeringId 는 신청한 학생과 대상 강좌를 연결한다. 이 그림은 유스케이스 1 과 2 에 해당하는 부분 모델이므

로 다른 유스케이스에서 사용하는 AcademicTerm 엔티티와 WaitlistEntry 엔티티는 표시하지 않았다.

배포 설계에는 애플리케이션과 PostgreSQL 컨테이너, TCP 5432 연결 및 10 GiB 저장소가 구성 조건으로 전달된다. 시스템은 이 조건을 가상머신 배치와 AWS 리소스 구성으로 구체화한다. 배포 다이어그램 전체 요소를 한 그림에 배치하면 요청 경로, 이미지 전달, 암호 처리와 리소스 생성 관계가 겹치므로 본 절에서는 같은 배포 설계를 기능별 다섯 관점으로 나누어 제시한다.

표 13. 생성된 AWS 배포 자원

ID	이름	종류	처리	Terraform 타입
registry-course-registration-app	Amazon ECR Repository	app-registry	create	aws_ecr_repository
registry-pull-identity-compute-1	IAM Role / VM Registry Pull	registry-pull-identity	create	aws_iam_role
registry-instance-profile-compute-1	IAM Instance Profile	registry-instance-profile	create	aws_iam_instance_profile
registry-pull-binding-compute-1-course-registration-app	IAM Role Policy Attachment	registry-pull-binding	create	aws_iam_role_policy_attachment
secret-ref-course-registration-app-course-registration-db-credential	Existing AWS Secret	secret-ref	referenceExisting	참조/외부 항목

ID	이름	종류	처리	Terraform 타입
secret-access-binding-course-registration-app-course-registration-db-credential	IAM Role Policy / State Secret Read	state-secret-access-binding	create	aws_iam_role_policy
secret-ref-course-registration-db-runtime-credential	Existing AWS Secret	secret-ref	referenceExisting	참조/외부 항목
secret-access-binding-course-registration-db-runtime-credential	IAM Role Policy / State Secret Read	state-secret-access-binding	create	aws_iam_role_policy
Network	VPC	network	create	aws_vpc
boot-image	Amazon Machine Image (AMI)	boot-image	referenceExisting	참조/외부 항목
internet-gateway	Internet Gateway	internet-gateway	create	aws_internet_gateway
public-route-table	Route Table / ingress	ingress-route-table	create	aws_route_table
public-default-route	Terraform: aws_route / Internet Gateway	ingress-default-route	create	aws_route
subnet-compute-1-1	Application Subnet	subnet	create	aws_subnet
route-association-compute-1-1	Terraform: aws_route_table_association / ingress	ingress-route-association	create	aws_route_table_association
traffic-filter-compute-1	Security Group / Application	security-group	create	aws_security_group
compute-1	EC2 Instance	compute-instance	create	aws_instance
public-ip-compute-1	Elastic IP	elastic-ip	create	aws_eip

ID	이름	종류	처리	Terraform 타입
data-disk-course-registration-db-data	EBS Volume	block-disk	create	aws_ebs_volume
disk-attachment-course-registration-db-data	Terraform: aws_volume_attachment	disk-attachment	create	aws_volume_attachment

표 13 는 애플리케이션 실행에 필요한 AWS 리소스 전체를 정리한 것이다. 그림 18 부터 그림 22 까지는 이 리소스를 외부 요청 처리, 데이터 저장, 이미지 전달, 암호 처리와 생성 순서로 나누어 보여 준다.

Public request and network ingress

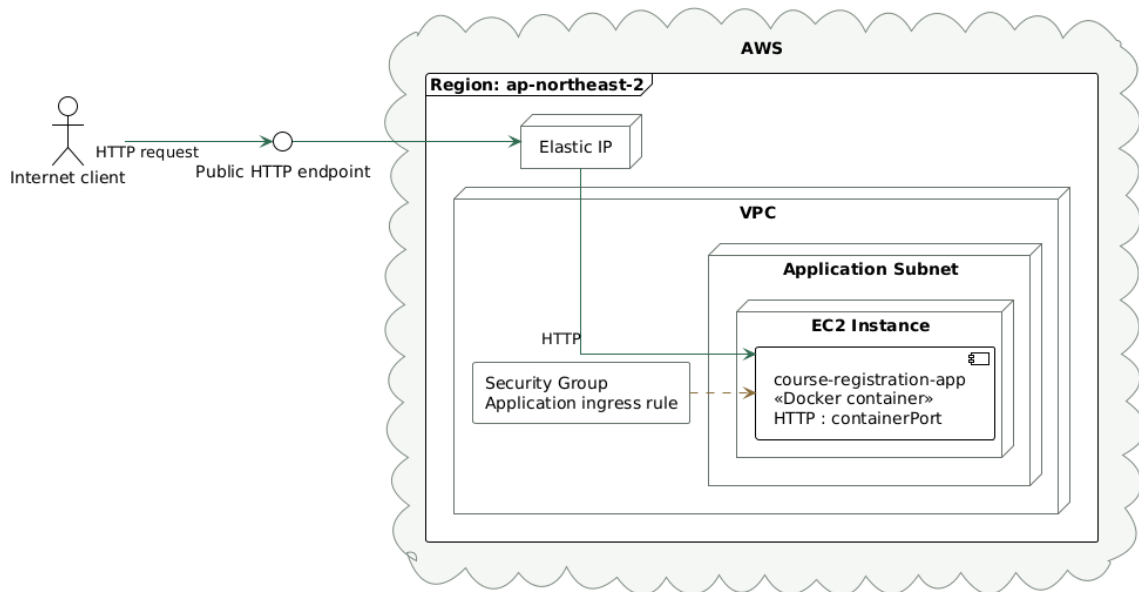


그림 18. 외부 요청 처리와 AWS 네트워크 진입 경로

그림 18 는 인터넷 사용자의 HTTP 요청이 공개 엔드포인트와 Elastic IP 를 거쳐 EC2 의 애플리케이션 컨테이너에 도달하는 경로를 나타낸다. EC2 는 AWS 서울 리전의 VPC 와 애플리케이션 서브넷에 배치되며, Security Group 은 애플리케이션 포트로 들어오는 요청을 허용한다.

Application, database, and persistent storage

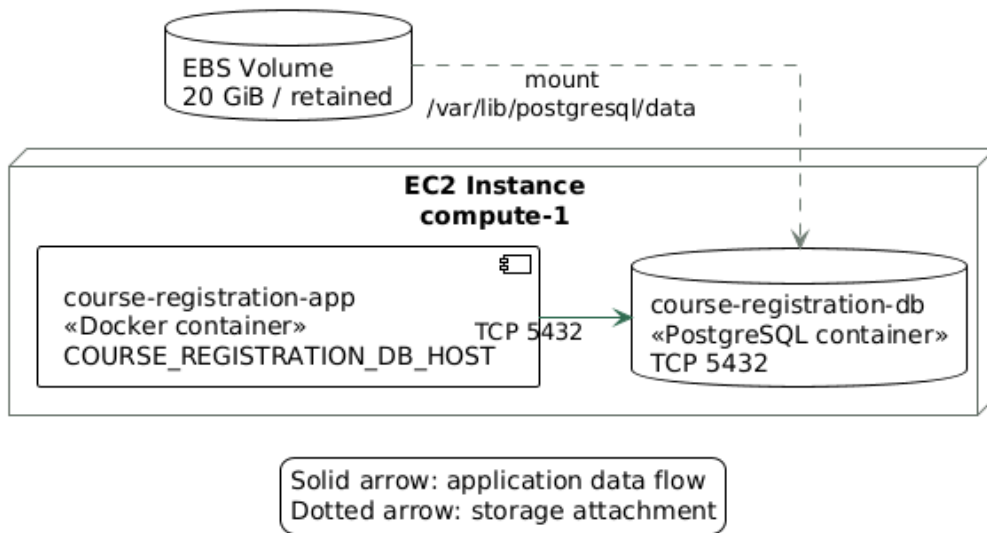


그림 19. 애플리케이션-데이터베이스 연결 및 영구 저장 구성

그림 19 는 한 EC2 인스턴스에서 실행되는 애플리케이션 컨테이너와 PostgreSQL 컨테이너의 연결을 보여 준다. 애플리케이션은 **COURSE_REGISTRATION_DB_HOST** 설정을 사용해 TCP 5432 로 데이터베이스에 접속한다. 10 GiB EBS 볼륨은 **/var/lib/postgresql/data** 에 마운트 되며, 삭제 시에도 데이터를 보존하도록 설정된다.

Container image build and delivery

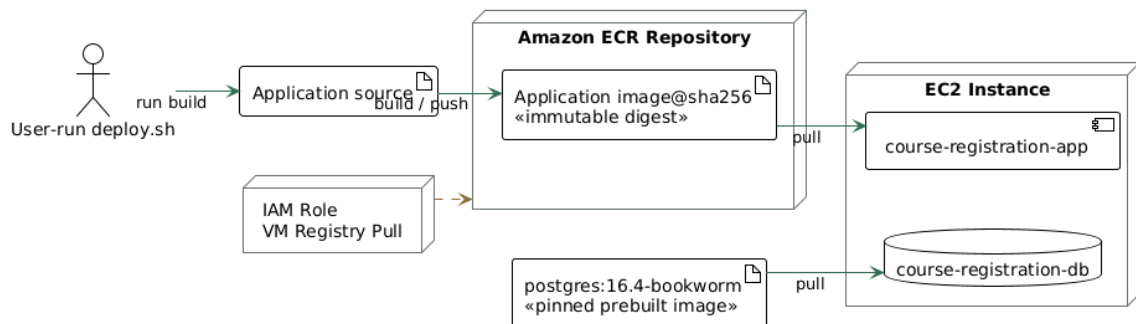


그림 20. 컨테이너 이미지 빌드 및 EC2 전달 과정

그림 20 는 배포 스크립트가 애플리케이션 소스에서 컨테이너 이미지를 빌드하고 Amazon ECR 에 푸시한 뒤, EC2 가 고정된 이미지 다이제스트를 기준으로 내려받는 과정을 나타낸다. EC2 에는 ECR 이미지를 읽을 수 있는 IAM 역할이 연결된다. PostgreSQL 컨테이너는 버전과 다이제스트가 고정된 사전 빌드 이미지를 사용한다.

Secret access and runtime injection

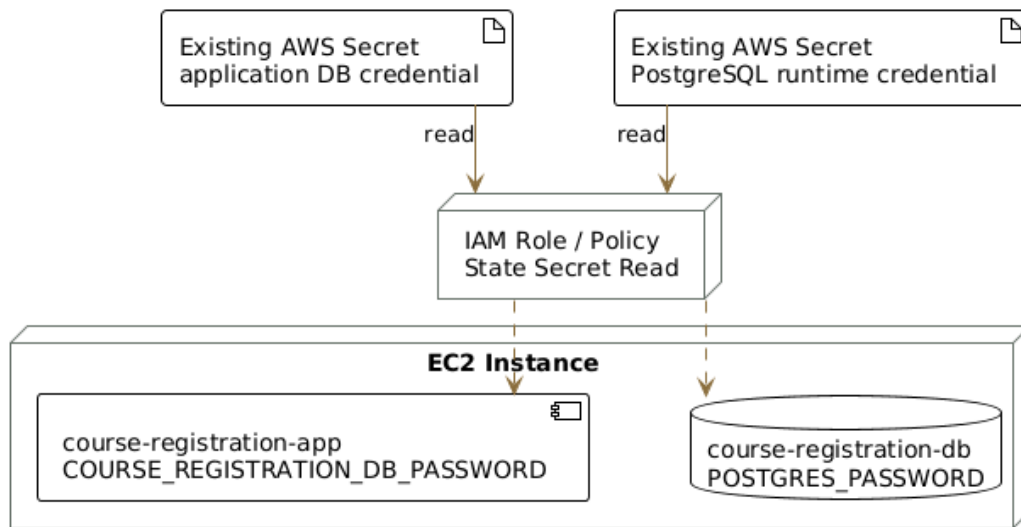


그림 21. AWS 암호 조회 및 컨테이너 환경 변수 주입

그림 21 는 애플리케이션용 데이터베이스 자격 정보와 PostgreSQL 실행 자격 정보를 AWS 의 암호에서 읽는 구조를 보여 준다. IAM 역할과 정책은 해당 암호에 대한 읽기 권한을 제공한다. EC2 초기화 과정은 조회한 값을 애플리케이션의 COURSE_REGISTRATION_DB_PASSWORD 와 데이터베이스의 POSTGRES_PASSWORD 환경 변수로 전달한다.

Terraform provisioning and association order

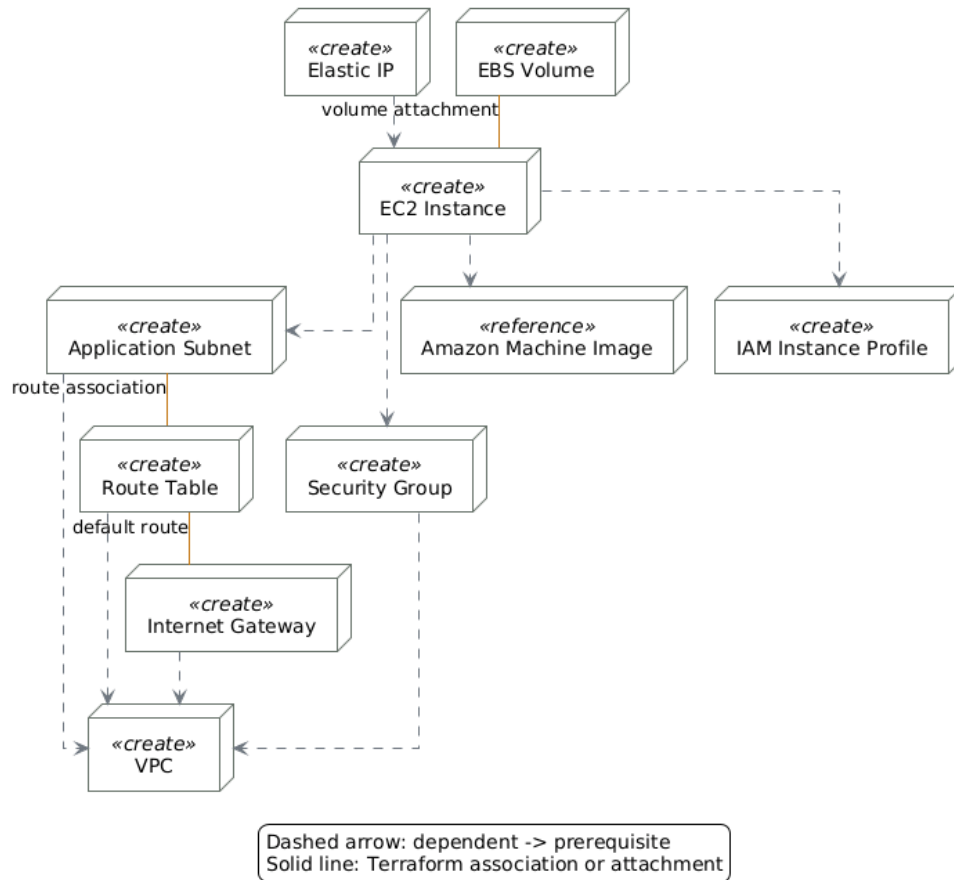


그림 22. Terraform 리소스 생성 및 연결 순서

그림 22 는 Terraform 이 네트워크, EC2 와 저장소를 구성할 때 사용하는 의존 관계를 나타낸다. VPC 는 인터넷 게이트웨이, 라우팅 테이블, 서브넷과 Security Group 의 선행 조건이다. EC2 는 서브넷, Security Group, AMI 와 IAM 인스턴스 프로파일을 참조하며, Elastic IP 와 EBS 볼륨은 EC2 에 연결된다. 점선 화살표는 의존 대상에서 선행 리소스를 가리키고 실선은 Terraform 의 연결 또는 부착 관계를 뜻한다.

수강신청 애플리케이션의 초기 클래스 설계에서 CourseOffering 클래스는 수강 등록이 추가될 때 incrementEnrolledCount()를 호출하지만, 등록이 제거되는 흐름에는 대응하는 감소 메서드가 존재하지 않았다. 이 상태에서는 수강 교환 또는 수강 취소 후 실제 등록 건수와 enrolledCount 가 달라질 수 있으므로, 충분히 사용자의 피드백 사항을 적용할 수 있다고 판단하였다. 따라서, 이에 대응하는 수강 등록 감소 메서드를 추가하기 위해 다음 표 14 와 같이 자연어 피드백을 전달하였다.

표 14. 수강 등록 감소 메서드 추가를 위해 전달된 사용자 피드백

구분	내용
실제 입력 원문	CourseOffering increases the enrollment count when a registration is added, but it does not decrease the count when a registration is removed. Add the corresponding decrease operation and apply it to every relevant removal flow. Preserve the existing increment behavior.
한국어 번역	CourseOffering 는 수강 등록이 추가될 때 등록 인원을 증가시키지만, 등록이 제거될 때는 인원을 감소시키지 않는다. 이에 대응하는 인원 감소 연산을 추가하고, 관련된 모든 등록 제거 흐름에 적용하라. 기존 인원 증가 동작은 유지하라.

시스템 설계 에이전트는 사용자 문장에 직접 등장한 CourseOffering 을 검색어를 이용해 현재 클래스 모델과 유스케이스 명세를 조회하고, 요구사항 추적표에 기록된 유스케이스의 시나리오들을 확인하였다. 에이전트는 이 근거로 수정 대상을 CourseOffering, 유스케이스 3 인 Swap Course Registration, 유스케이스 5 인 Drop Course Registration 으로 확장하였다. 이어서, 다음 표 15 와 같이 하나의 피드백을 세 개의 지시로 분해하였다.

표 15. 사용자 피드백 분해 결과

단락	원문	한국어 번역
CourseOffering 클래스	Add a new operation decrementEnrolledCount() with no parameters and return type void to the CourseOffering class.	CourseOffering 클래스에 매개변수가 없고 반환형이 void 인 새 연산 decrementEnrolledCount()를 추가한다.
유스케이스 3 Swap Course Registration	Insert a call to CourseOffering::decrementEnrolledCount() at step 4 (original offering) of the atomic swap flow in UC3.	UC3 의 원자적 수강 교환 흐름에서 4 단계의 기존 강좌에 CourseOffering::decrementEnrolledCount() 호출을 삽입한다.
유스케이스 5 Drop Course Registration	Insert a call to CourseOffering::decrementEnrolledCount() at step 3 (offering of the dropped registration) of the drop registration flow in UC5.	UC5 의 수강 취소 흐름에서 3 단계의 취소 대상 등록과 연결된 강좌에 CourseOffering::decrementEnrolledCount() 호출을 삽입한다.

에이전트는 분해한 세 개의 지시 중 CourseOffering 클래스 구조를 먼저 처리하고, 다음으로 유스케이스 3 과 5 의 호출 트리를 수정하였다. 수정 이후, 추가된 메서드는 요구사항 추적표에 반영되며, 기존의 incrementEnrolledCount()와 같이 유지되어야 하는

메서드들 및 추적 사항은 그대로 유지되었다. 또한, 유스케이스 5 의 수정된 호출 트리에는 권한 검사 대상 학생을 찾는 Student::findById() 호출도 함께 추가되었음을 관찰할 수 있었다. 다음 그림 23 부터 그림 25 까지는 피드백 적용 전·후의 클래스 및 시퀀스 다이어그램 변경사항을 나타낸다.

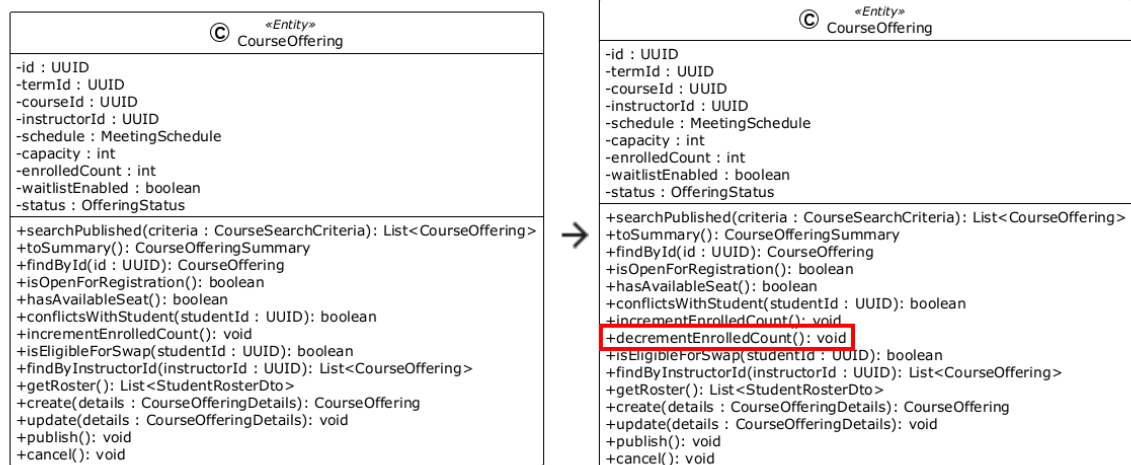


그림 23. 피드백 적용 전·후 CourseOffering 클래스 다이어그램 비교

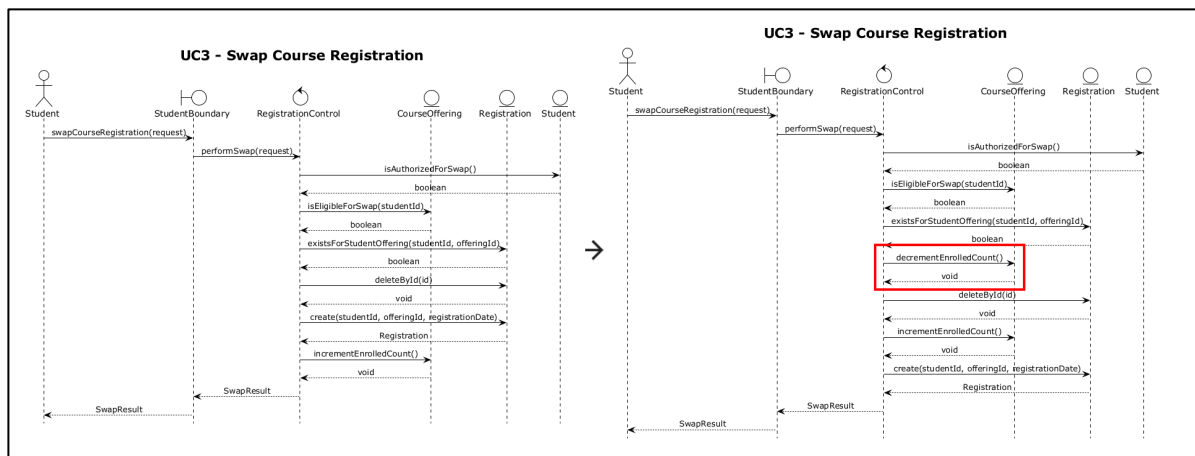


그림 24. 피드백 적용 전·후 유스케이스 3 의 시퀀스 다이어그램 비교

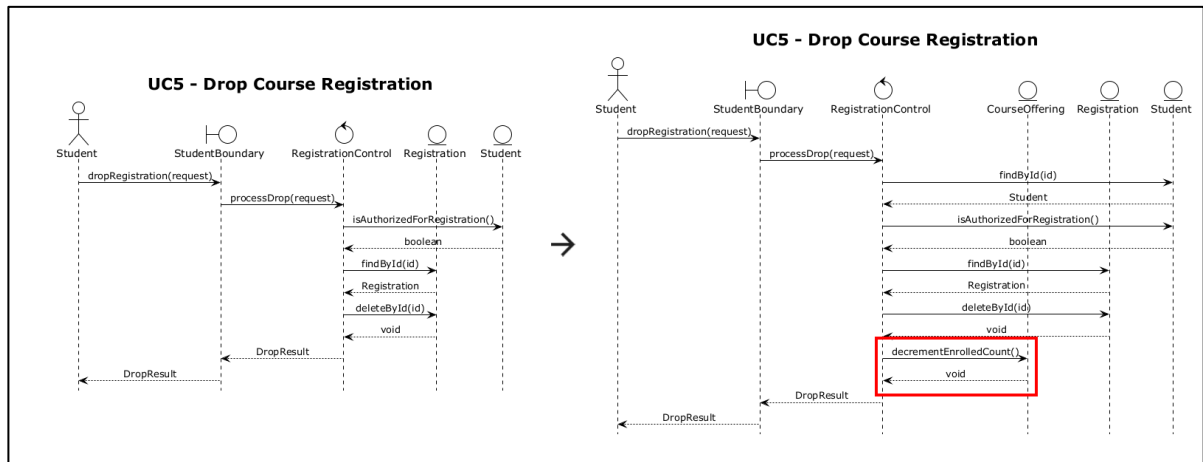


그림 25. 피드백 적용 전·후 유스케이스 5의 시퀀스 다이어그램 비교

4.3. 시스템 구현 에이전트

본 절에서는 설계 에이전트가 만든 클래스·시퀀스·ERD·OpenAPI 명세와 배포 설계를 구현 에이전트에 전달하여 소스 코드와 실행·배포 파일을 생성한 결과를 설명한다. 먼저 백엔드 코드를 생성하고, 이어서 같은 OpenAPI 명세를 사용하는 프론트엔드 코드를 생성하였다. 두 소스가 준비된 뒤 컨테이너 빌드·실행 파일을 구성하고, 확인된 실행 설정을 배포 설계에 연결하여 IaC 코드를 생성하였다. 전체 구현 산출물은 172 개 파일이다.

표 16. 시스템 구현 단계에서 생성된 파일 구성

구분	파일 수	생성 내용
백엔드 빌드·실행 설정	8	Gradle 설정, 실행 진입 코드, 보안·데이터베이스 설정
백엔드 API 처리 코드	28	API 선언 파일 4 개, 요청·응답 데이터 모델 20 개, 컨트롤러 4 개
BCE 설계 코드 및 기능 처리 서비스	40	BCE 설계 클래스 34 개와 기능 처리 서비스 6 개
데이터 저장	8	JPA 엔티티 4 개와 데이터 접근용 Repository 4 개
백엔드 테스트	2	애플리케이션 구동 테스트와 테스트 환경 설정
프론트엔드 소스·설정	14	React 소스 6 개와 Vite·TypeScript·npm 설정 8 개
자동 생성된 API 호출 코드·문서	56	TypeScript API 호출 코드, 데이터 모델, 실행 지원 코드와 문서
웹 빌드·실행 및 배포 파일	16	웹 빌드 파일 3 개와 Docker Compose·IaC 배포 파일 13 개
합계	172	구현 산출물에 포함된 전체 파일

백엔드와 프론트엔드는 서로 다른 에이전트가 작성하지만 같은 OpenAPI 명세를 기준으로 삼는다. API 경로와 요청·응답 데이터 형식을 먼저 정한 뒤, 백엔드 컨트롤러와 프론트엔드 API 호출 코드가 같은 작업 식별자(operationId)와 데이터 모델을 사용하도록 하였다.

그림 26 은 생성된 백엔드의 StudentsApiController Java 소스 코드에서 수강 신청 요청을 처리하는 부분이다. 컨트롤러는 학생 식별자를 경로 변수로 받고 요청 본문을 RegisterRequest 클래스로 검증한 뒤 RegistrationControl 클래스에 처리를 위임한다. 처리 결과는 RegistrationOutcome 응답 클래스로 변환하여 HTTP 201 응답으로 반환한다.

```
@RequestMapping(
    method = RequestMethod.POST,
    value = StudentsApi.PATH_REGISTER_FOR_COURSE_OFFERING,
    produces = { "application/json" },
    consumes = { "application/json" }
)
@Override
public ResponseEntity<RegistrationOutcome> registerForCourseOffering(
    @Parameter(name = "studentId", description = "", required = true, in = ParameterIn.PATH) @PathVariable("studentId") String studentId,
    @Parameter(name = "RegisterRequest", description = "", required = true) @Valid @RequestBody RegisterRequest registerRequest
) {
    var result = registrationControl.register(objectMapper.convertValue(registerRequest, com.easydep.app.bce.RegisterRequest.class));
    var response = objectMapper.convertValue(result, RegistrationOutcome.class);
    return ResponseEntity.status(201).body(response);
}
```

그림 26. 백엔드 수강 신청 API 요청 처리 코드

표 17 에 나타낸 바와 같이 백엔드는 Spring Boot 실행 코드, API 요청 처리 코드, 기능 처리 서비스와 데이터 저장 코드로 구성되었다. OpenAPI 명세를 바탕으로 API 경로와 요청·응답 형식을 선언한 Java 파일과 데이터 모델 20 개를 생성하였다. 컨트롤러는 요청을 기능 처리 서비스에 전달하고, ERD 의 주요 엔티티에는 JPA 클래스와 데이터 접근용 Repository 를 연결하였다.

표 17. 생성된 백엔드 구현 산출물

계층	대표 파일	확인한 생성 결과
애플리케이션	Spring Boot 애플리케이션 진입 클래스	Spring Boot 실행 진입점 생성
API 명세	OfferingsApi.java 외 3 개	API 기능 14 개의 경로·메서드와 요청·응답 형식 선언
API 요청 처리	StudentsApiController.java 외 3 개	API 요청 처리 코드와 기능 처리 서비스 연결
기능 처리	RegistrationControlService.java 외 5 개	수강 신청·교환·취소, 시간표, 대기열과 학사 관리 서비스 생성
데이터 저장	RegistrationRepository.java 외 7 개	JPA 엔티티 4 개와 데이터 접근용 Repository 4 개 생성
데이터베이스 초기화	V1_Initial_Schema.sql	ERD 를 반영한 초기 데이터베이스 생성 SQL 작성
테스트	BackendApplicationTest.java	Spring 애플리케이션 구동 설정 확인용 테스트 생성

백엔드 구현 작업은 필수 출력 11 개를 생성한 뒤 Gradle 테스트를 통과하였다. 수강 신청 API 는 RegistrationControl 클래스의 register 메서드와 연결되어 HTTP 201 응답을 반환한다. 구현 단계의 검사는 파일 생성과 빌드 성공을 확인한 것이므로 신청 조건과 데이터 변경이 요구사항대로 동작하는지는 테스트 단계에서 API 를 실행하여 확인한다.

그림 27 은 UI 의 api.ts 에서 생성된 API 클라이언트를 연결한 부분이다. API_BASE_URL 로 실행 환경별 서버 주소를 설정하고, DefaultApi 가 제공하는 14 개 API 호출을 apiCalls 객체에 모아 화면 코드가 같은 방식으로 사용하도록 구성하였다.

```
const defaultApi = new DefaultApi(new Configuration({ basePath: API_BASE_URL }));

export const apiCalls = {
  cancelWaitlist: (request: CancelWaitlistRequest) => defaultApi.cancelWaitlist(request),
  dropRegistration: (request: DropRegistrationRequest) => defaultApi.dropRegistration(request),
  exportSchedule: (request: ExportScheduleRequest) => defaultApi.exportSchedule(request),
  manageCourseOffering: (request: ManageCourseOfferingOperationRequest) => defaultApi.manageCourseOffering(request),
  registerForCourseOffering: (request: RegisterForCourseOfferingRequest) => defaultApi.registerForCourseOffering(request),
  requestCourseOfferingSearch: (request: RequestCourseOfferingSearchRequest) => defaultApi.requestCourseOfferingSearch(request),
  requestJoinWaitlist: (request: RequestJoinWaitlistOperationRequest) => defaultApi.requestJoinWaitlist(request),
  requestSwap: (request: RequestSwapOperationRequest) => defaultApi.requestSwap(request),
  selectCourseOffering: (request: SelectCourseOfferingRequest) => defaultApi.selectCourseOffering(request),
  submitTerm: (request: SubmitTermRequest) => defaultApi.submitTerm(request),
  viewAssignedCourseOfferings: (request: ViewAssignedCourseOfferingsRequest) => defaultApi.viewAssignedCourseOfferings(request),
  viewCourseOfferingDetails: (request: ViewCourseOfferingDetailsRequest) => defaultApi.viewCourseOfferingDetails(request),
  viewCurrentRegistrations: (request: ViewCurrentRegistrationsRequest) => defaultApi.viewCurrentRegistrations(request),
  viewOwnWaitlistEntries: (request: ViewOwnWaitlistEntriesRequest) => defaultApi.viewOwnWaitlistEntries(request),
};
```

그림 27. UI API 호출 연결 코드

표 18 에 나타난 바와 같이 UI 는 React 와 Vite 로 구성되며, OpenAPI Generator 가 만

은 TypeScript API 호출 코드와 데이터 모델을 포함한다. 생성 클라이언트에는 백엔드 API 기능 14개에 대응하는 호출 메서드와 데이터 모델 20개가 포함되었다. npm 빌드 결과 HTML, JavaScript, CSS 배포 파일이 생성되었다.

표 18. 생성된 UI 구현 산출물

구분	대표 파일	확인한 생성 결과
화면 시작 코드	App.tsx, main.tsx	React 애플리케이션을 시작하는 기본 코드 생성
API 연결	api.ts, DefaultApi.ts	환경별 API 주소 설정과 OpenAPI 명세를 반영한 호출 코드 생성
데이터 모델	generated/src/models/*.ts	AcademicTerm, CourseOffering, Registration 등 20개 모델 생성
프로젝트 설정	package.json, vite.config.ts	개발 서버 실행·빌드 명령과 TypeScript 설정 생성
빌드 결과	dist/index.html 외 2개	웹 배포용 HTML·JavaScript·CSS 파일 생성

프론트엔드 구현 작업과 npm 빌드는 성공하였고 39개 모듈을 처리하여 배포 파일 3개를 생성하였다. API 호출 코드는 환경별 서버 주소와 OpenAPI 기반 요청 형식을 사용한다. 화면과 API의 실제 연결 및 사용자 입력에 따른 상태 변화는 테스트 단계에서 확인한다.

구현이 끝난 뒤 생성된 소스를 컨테이너 이미지로 묶기 위해 Dockerfile 과 .dockerignore 를 생성하였다. 그림 28의 Dockerfile 은 Node.js 단계에서 프론트엔드를 빌드하고, Gradle 단계에서 Spring Boot 실행 파일을 만든다. JRE 21 기반 최종 이미지에는 실행 파일과 프론트엔드 정적 파일만 포함된다. 컨테이너는 비루트 사용자로 8000번 포트에서 실행되며, 이 실행 설정은 뒤이어 생성되는 IaC 코드와 VM 초기화 스크립트에 반영된다.

```

FROM node:20-alpine AS frontend-build
WORKDIR /app/frontend
COPY frontend/package.json frontend/package-lock.json ./
RUN npm ci --ignore-scripts --no-audit --no-fund \
    && test -x node_modules/.bin/tsc \
    && test -x node_modules/.bin/vite
COPY frontend/ ./
ARG VITE_API_BASE_URL=""
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
RUN npm run build

FROM gradle:8.14.2-jdk21 AS build
WORKDIR /app
COPY . .
COPY --from=frontend-build /app/frontend/dist/ src/main/resources/static/
RUN gradle bootJar --no-daemon \
    && jar="$(find build/libs -maxdepth 1 -type f -name '*.jar' ! -name '*-plain.jar' -print | sort | head -n 1)" \
    && test -n "$jar" \
    && cp "$jar" /tmp/app.jar

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
# Use the same numeric identity as the VM bootstrap when it owns persistent paths.
RUN addgroup -S -g 10001 app && adduser -S -D -H -u 10001 -G app app
COPY --chown=10001:10001 --from=build /tmp/app.jar app.jar
USER 10001:10001
# Keep the Spring Boot listen port aligned with the container's published port.
ENV SERVER_PORT=8000
EXPOSE 8000
ENTRYPOINT ["java", "-jar", "app.jar"]

```

그림 28. 생성된 Dockerfile 의 멀티 스테이지 빌드 구성

Dockerfile 에서 확인한 실행 포트와 사용자 설정을 시스템 설계 에이전트가 생성한 AWS 배포 모델에 연결하여 Terraform 문법의 IaC 코드와 실행 보조 스크립트를 생성하였다. main.tf 는 VPC, 서브넷, 보안 그룹, EC2, EBS, Elastic IP, ECR, IAM 과 각 리소스의 연결 관계를 정의한다. variables.tf 와 outputs.tf 는 배포 입력값과 생성 후 확인할 주소를 분리하며, VM 초기화 템플릿은 Docker 실행 환경과 데이터 볼륨을 준비한다. deploy.sh, plan.sh, status.sh 등의 스크립트는 배포 계획 확인, 적용, 상태 조회를 지원한다.

표 19. 생성된 주요 IaC 코드 및 배포 실행 파일

파일	역할	크기(byte)
bootstrap_compute_1.sh.tpl	VM 에서 Docker Compose 와 EBS 마운트를 준비하는 cloud-init 템플릿	3013
deploy.sh	초기화, 계획 검토와 적용 실행	1057
destroy.sh	생성 리소스 제거	99
doctor.sh	로컬 도구와 인증 사전 검사	176
제안하는 시스템-provider.tf	Terraform 및 AWS provider 설정	385
main.tf	AWS 네트워크, EC2, ECR, IAM, EBS 와 연결 리소스	5136

파일	역할	크기(byte)
outputs.tf	배포 후 공개 주소와 리소스 식별자 출력	130
plan.sh	Terraform 계획 생성	140
status.sh	배포 상태와 출력 조회	99
variables.tf	VM 유형, 키, 포트, 이미지 digest 와 Secret 참조 입력	1070

표 19 는 생성된 IaC 코드와 배포 실행 파일을 역할별로 정리한 것이다.

variables.tf 는 가상머신 유형, SSH 공개 키, 부팅 이미지, 컨테이너 이미지 다이제스트, Secrets Manager 참조 값을 배포 변수로 선언한다. 그림 29 은 파일의 앞부분을 실제 편집기 화면으로 보여 준다.

```
variable "resource_prefix" {
  type    = string
  default = "easydep"
}

variable "runtime_env" {
  type      = string
  default   = ""
  sensitive = true
}

variable "vm_sku" {
  type    = string
  default = "t3.small"
}

variable "offline_validation" {
  type      = bool
  default   = false
  description = "Internal switch used only by the offline template test."
}
```

그림 29. variables.tf 의 배포 변수 선언 일부

```
variable "resource_prefix" {
  type    = string
  default = "제한하는 시스템"
}

variable "runtime_env" {
  type      = string
  default   = ""
  sensitive = true
}

variable "vm_sku" {
  type    = string
  default = "t3.small"
}

variable "offline_validation" {
  type      = bool
  default   = false
```

```

description = "Internal switch used only by the offline template test."
}

variable "ssh_public_key" {
  type      = string
  sensitive = true
  default   = ""
}

variable "boot_image_id" {
  type = string
  validation {
    condition = startswith(var.boot_image_id, "ami-")
    error_message = "boot_image_id must be an explicit AMI id."
  }
}

variable "image_digest_application" {
  type = string
  validation {
    condition = startswith(var.image_digest_application, "sha256:")
    error_message = "image_digest_application must be an immutable sha256 digest."
  }
}

variable "secret_reference_application_security_password" {
  type      = string
  sensitive = true
  validation {
    condition = can(regex("^arn:(aws|aws-us-gov|aws-cn):secretsmanager:[a-z0-9-]+:[0-9]{12}:secret:[A-Za-z0-9/_+=.@-]+$", var.secret_reference_application_security_password))
    error_message = "secret_reference_application_security_password must be a Secrets Manager ARN; secret values are never accepted here."
  }
}

```

main.tf 에는 표 13 에서 정리한 배포 구조를 AWS 리소스로 선언하였다. 그림 30 은 VPC, 인터넷 게이트웨이, 라우팅 테이블, 기본 경로, 서브넷으로 이어지는 네트워크 구성 부분이다. 그림에 담기지 않은 선언도 확인할 수 있도록 아래 코드에는 main.tf 에 포함된 16 개 AWS 리소스 선언을 모두 제시하였다.

```

resource "aws_vpc" "network" {
  cidr_block      = "10.80.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
}

resource "aws_internet_gateway" "internet_gateway" {
  vpc_id = aws_vpc.network.id
}

resource "aws_route_table" "public_route_table" {
  vpc_id = aws_vpc.network.id
}

resource "aws_route" "public_default_route" {
  route_table_id      = aws_route_table.public_route_table.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id          = aws_internet_gateway.internet_gateway.id
}

resource "aws_subnet" "subnet_compute_1_1" {
  vpc_id            = aws_vpc.network.id
  cidr_block        = "10.80.1.0/24"
  map_public_ip_on_launch = false
  availability_zone  = "ap-northeast-2a"
}

```

그림 30. main.tf 의 AWS 네트워크 리소스 선언 일부

```

resource "aws_ecr_repository" "registry_application" {
resource "aws_iam_role" "registry_pull_identity_compute_1" {
resource "aws_iam_instance_profile" "registry_instance_profile_compute_1" {
resource "aws_iam_role_policy_attachment" "registry_pull_binding_compute_1_application" {
resource "aws_iam_role_policy" "secret_access_binding_application_security_password" {
resource "aws_vpc" "network" {
resource "aws_internet_gateway" "internet_gateway" {
resource "aws_route_table" "public_route_table" {
resource "aws_route" "public_default_route" {
resource "aws_subnet" "subnet_compute_1_1" {
resource "aws_route_table_association" "route_association_compute_1_1" {
resource "aws_security_group" "traffic_filter_compute_1" {
resource "aws_instance" "compute_1" {
resource "aws_eip" "public_ip_compute_1" {
resource "aws_ebs_volume" "data_disk_workload_data" {
resource "aws_volume_attachment" "disk_attachment_workload_data" {

```

월 5 달러 조건은 배포 설계 입력에 보존되어 있으나 생성 결과에는 예상 비용 합계가 포함되지 않았다. 따라서 배포 전 계획 결과와 AWS 요금 정보를 이용한 비용 확인이 필요하다. IaC 코드의 정적 검사와 실제 AWS 배포 및 애플리케이션 응답 검증은 뒤 절에서 구분하여 제시한다.

구현 결과는 필수 파일 생성, 백엔드:프론트엔드 빌드, 설계 산출물과 생성 코드의 대응 관계, IaC 정적 검사를 기준으로 확인하였다. 구현 완료 기록에는 필수 출력 14 개가 모두 생성되었고, 누락된 출력과 남은 구현 작업은 각각 0 건으로 기록되었고, 백엔드 Gradle 테스트와 프론트엔드 npm 빌드는 종료 코드 0 으로 완료되었다. 설계 대응 관계

검사도 대상 58 개에서 위반 0 건으로 통과하였으며, IaC 코드와 VM 초기화·실행 보조 파일의 정적 검사 결과도 통과로 기록되었다.

표 20. 시스템 구현 단계별 검증 결과

검증 항목	판정	확인 결과	다음 단계
필수 파일 생성	통과	필수 출력 14 개 생성, 누락 0 건, 남은 작업 0 건	빌드 및 설계 대응 관계 확인
백엔드 구현	통과	필수 출력 11 개 생성, Gradle 테스트 종료 코드 0	테스팅 단계에서 API 기능 확인
프론트엔드 구현	통과	필수 출력 3 개 생성, npm 빌드 종료 코드 0	테스팅 단계에서 화면·API 연결 확인
IaC 생성	통과	IaC 코드와 VM 초기화·실행 보조 파일 정적 검사 통과	배포 후 리소스와 응답 확인
설계 대응 관계	통과	추적 대상 58 개 확인, 위반 0 건	동적 결과와 함께 최종 판단
테스트 단계 전달	준비 완료	소스 코드·테스트·실행 및 배포 파일을 테스트 입력으로 구성	정적 검사와 동적 API 검사 수행

표 20 은 이러한 구현 산출물의 생성과 정적 유효성을 확인한 결과를 나타낸다. 구현 단계는 필수 파일 생성, 빌드, 설계 대응 관계와 IaC 정적 검사를 완료하여 테스트 단계에 전달할 산출물을 마련하였다.

4.4. 테스트 에이전트

테스팅 에이전트는 구현 산출물을 실행해 API 동작을 확인하는 동적 테스팅과, 소스 코드를 실행하지 않고 배포 파일의 구성과 문법을 확인하는 정적 테스팅을 수행하였다. 두 검사에는 서로 다른 도구를 사용했으며 결과도 구분하여 기록하였다. 자동 테스팅을 마친 뒤에는 같은 Dockerfile 과 IaC 코드를 AWS 서울 리전에 배포하여 실제 클라우드 환경에서 실행되는지도 확인하였다.

동적 테스팅은 유스케이스 1 인 강좌 검색과 유스케이스 2 인 수강 신청을 대상으로 수행하였다. 각 테스트는 기능 수행에 필요한 API 호출 순서, 입력값과 예상 응답을 포함한다. 아래에는 두 테스트에 사용한 계획과 실행 결과의 실제 값을 발췌하였다.

유스케이스 1 의 테스트는 먼저 개설 강좌 목록을 조회하고, 조회 결과의 첫 번째 강좌 식별자를 다음 상세 조회에 전달하도록 구성되었다. 목록 조회에는 requestCourseOfferingSearch API 를, 상세 조회에는 viewCourseOfferingDetails API 를 사용하였다. 첫 번째 요청인 GET /offerings 는 HTTP 200 을 반환하여 API 형식 검사를

통과했지만 응답 본문은 빈 목록이었다. 이 때문에 다음 단계에서 첫 번째 강좌의 식별자를 구할 수 없어 상세 조회는 실행되지 않았다. 실패 원인은 애플리케이션 오류가 아니라 빈 검색 결과를 처리하지 않은 테스트 계획 문제로 판정되었다.

다음 코드는 테스트 작업 결과에 보존된 유스케이스 1 의 Arazzo 계획과 실행 결과이다. 첫 단계의 응답 전체를 offeringsList로 저장하고, 두 번째 단계가 그 목록의 첫 항목에 있는 id를 참조하도록 작성된 부분과 실제 빈 목록 응답을 확인할 수 있다.

```
[Arazzo 테스트 계획]
{
  "workflowId": "workflow-UC1",
  "steps": [
    {
      "stepId": "search",
      "operationId": "requestCourseOfferingSearch",
      "outputs": {"offeringsList": "$response.body"},
      "successCriteria": [{"condition": "$statusCode == 200"}]
    },
    {
      "stepId": "viewOfferingDetails",
      "operationId": "viewCourseOfferingDetails",
      "parameters": [{
        "in": "path",
        "name": "offeringId",
        "value": "$steps.search.outputs.offeringsList#/0/id"
      }],
      "successCriteria": [{"condition": "$statusCode == 200"}]
    }
  ]
}

[실행 결과]
{
  "workflowId": "workflow-UC1",
  "gateStatus": "FAIL",
  "steps": [
    {
      "stepId": "search",
      "method": "GET",
      "path": "/offerings",
      "statusCode": 200,
      "responseBody": "[]",
      "contractStatus": "PASS"
    },
    {
      "stepId": "viewOfferingDetails",
      "status": "failed",
      "message": "JSON Pointer does not resolve: #/0/id"
    }
  ]
}
```

유스케이스 2의 테스트는 학생 식별자를 주소 경로에 넣고 수강 신청 정보를 JSON 본문으로 전송한 뒤 HTTP 201 응답을 확인하도록 구성되었다. 테스트 계획은 registerForCourseOffering API를 선택하고 요청 본문에 offeringId를 넣었다. 그러나 실제 OpenAPI 명세는 같은 값의 필드명을 courseOfferingId로 정의하고 있었다. 테스트 에이전트는 요청을 보내기 전에 본문을 API 명세와 대조했고, 필수 항목이 없다고 판단하여 실행을 중단하였다. 따라서 유스케이스 2의 결과도 테스트 계획 오류이며 수강 신청 구현의 성

공이나 실패를 판정한 결과는 아니다.

다음 코드에는 유스케이스 2 의 Arazzo 계획과 입력 검사 결과를 제시하였다. 계획에 작성된 offeringId 와 검사 결과에서 요구한 courseOfferingId 를 비교하면 테스트가 API 호출 전에 중단된 이유를 확인할 수 있다.

```
[Arazzo 테스트 계획]
{
  "workflowId": "workflow-UC2",
  "steps": [{
    "stepId": "register",
    "operationId": "registerForCourseOffering",
    "parameters": [{
      "in": "path",
      "name": "studentId",
      "value": "student-1"
    }],
    "requestBody": {
      "payload": {"offeringId": "offering-1"},
      "contentType": "application/json"
    },
    "successCriteria": [{"condition": "$statusCode == 201"}]
  }]
}

[실행 결과]
{
  "workflowId": "workflow-UC2",
  "gateStatus": "FAIL",
  "failedStepId": "register",
  "defectClass": "TEST_DEFECT",
  "reason": "Input body is invalid: 'courseOfferingId' is a required property"
}
```

두 동적 테스트 모두 계획과 실행 결과가 테스트링 작업 기록 안에 보존되었다. 유스케이스 1 은 첫 번째 API 호출까지 수행했으며, 유스케이스 2 는 요청 본문 검사에서 중단되어 실제 API 호출 기록이 없다.

정적 테스트에서는 Trivy, cloud-init 검사 도구, 셸 구문 검사, Docker Compose 와 OpenTofu 를 사용하였다. Trivy 는 Dockerfile 과 AWS 인프라 설정에서 보안상 주의가 필요한 구성을 찾는다. 배포 패키지 검사는 cloud-init 설정 파일의 형식, Linux 셸 스크립트의 문법, Docker Compose 서비스 구성과 Windows PowerShell 스크립트의 문법을 확인하였다. IaC 코드는 OpenTofu 로 형식을 정리한 뒤 필요한 플러그인을 초기화하고 전체 구성의 유효성을 검사하였다.

Trivy 검사에서는 AWS 보안 그룹의 외부 통신 규칙과 관련된 AWS-0104 항목 2 건이 탐지되었다. 확인 결과 해당 규칙은 패키지 설치와 ECR 이미지 다운로드에 필요한 TCP 80-443 통신만 허용하고 있었으며, 모든 프로토콜을 외부로 개방한 구성은 아니었다. 이에 따라 두 항목은 배포에 필요한 허용 사항으로 처리되었고 정적 보안 검사 결과는 통과로 기록되었다. cloud-init 형식 검사, 셸 스크립트 문법 검사, Docker Compose 구성

검사와 PowerShell 문법 검사도 모두 통과하였다. OpenTofu 의 형식 검사, 초기화와 구성 검증 역시 종료 코드 0 으로 완료되었다.

4.5. 클라우드 리소스 배포



그림 31. VM 생성 의존성에 따라 배포된 리소스 구성의 일부

세부 정보

보안 그룹 이름 ed3135-dash0908-traffic_filter_comput e_1-20260908021503284300000003	보안 그룹 ID sg-0decbaee01ef02892	설명 Managed by Terraform	VPC ID vpc-080c7419bba05c41d
소유자 838062934141	인바운드 규칙 수 1 권한 항목	아웃바운드 규칙 수 3 권한 항목	

인바운드 규칙 | 아웃바운드 규칙 | 공유 | VPC 연결 | 관련 리소스 - 신규 | 태그

인바운드 규칙 (1)

보안 그룹 규칙 ID	IP 버전	유형	프로토콜	포트 범위	소스
sgr-02dbea27c7d0cdc75	IPv4	사용자 지정 TCP	TCP	8000	0.0.0.0/0

그림 32. 애플리케이션 포트 TCP 8000 을 허용한 EC2 보안 그룹

탄력적 IP 주소 (1) 정보

할당된 IPv4 주소	유형	할당 ID	역방향 DNS 레코드	연결된 인스턴스 ID
54.116.198.26	퍼블릭 IP	eipalloc-09a637d1006142700	-	i-0af8b4c91ccd74c05

그림 33. EC2 인스턴스에 연결된 Elastic IP

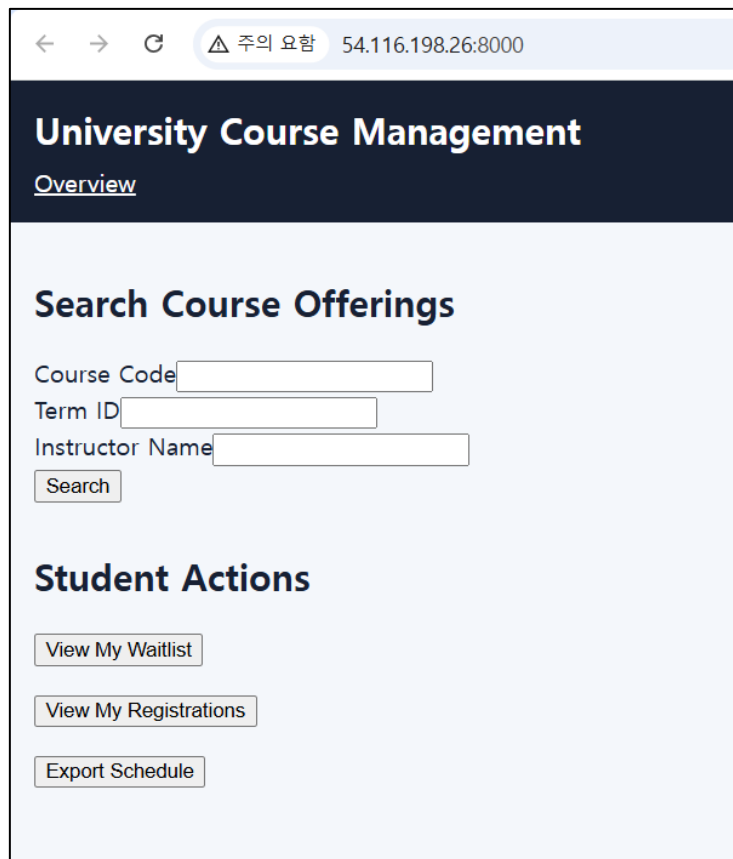


그림 36. 배포된 수강신청 애플리케이션의 메인 화면

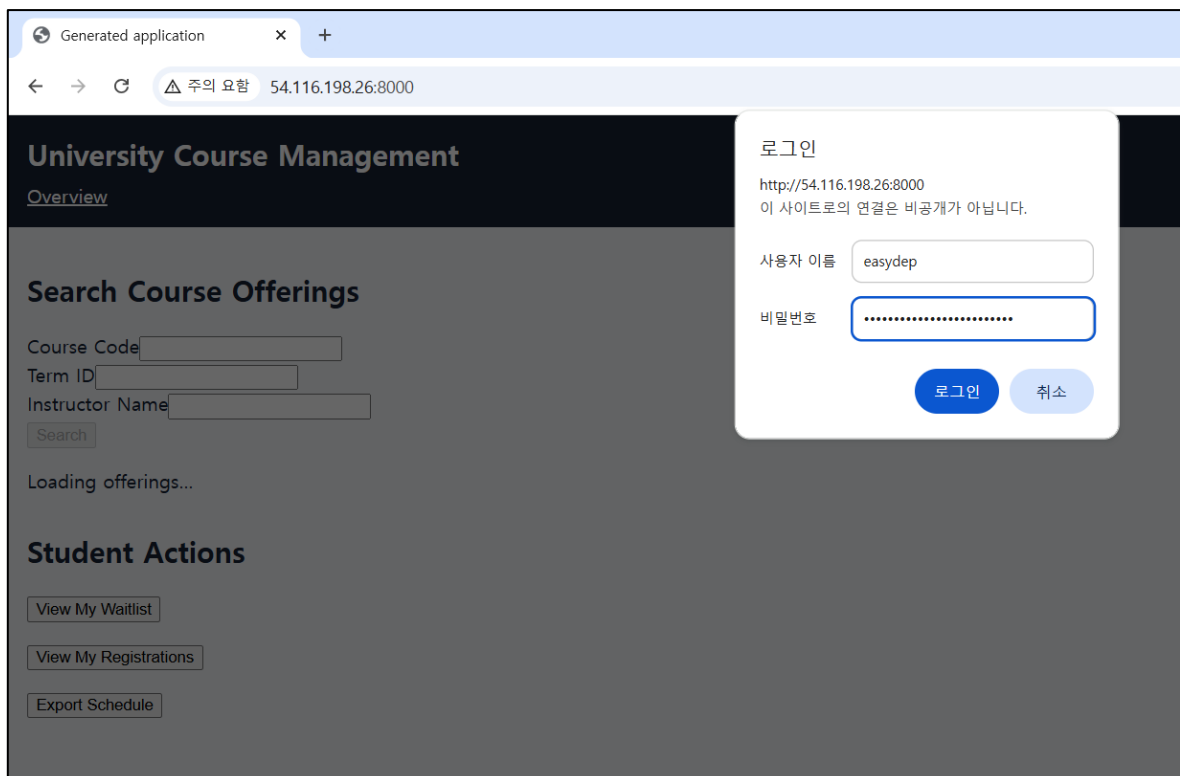


그림 37. 시험용 암호를 사용하는 브라우저 HTTP Basic 인증

그림 36 과 그림 37 에 따라, 본 AWS 배포 실험에서는 클라우드 리소스의 생성 및 애플리케이션 실행 의존성을 고려하여 애플리케이션이 성공적으로 배포 및 실행되었음을 관찰할 수 있었다.

5. 비교 분석 및 평가

5.1. 평가 대상과 방법

5.1.1. 평가 대상 및 비교 조건

본 연구에서는 제안하는 시스템의 개발 지원 성능을 공개 멀티 AI 에이전트 개발 지원 도구인 MetaGPT 와 ChatDev 를 기준선으로 하여 비교하도록 평가 방법을 설계하였다.

MetaGPT 는 표준 운영 절차(SOP, Standard Operating Procedure)에 따라 제품 관리자, 아키텍트, 엔지니어 등의 역할을 연결하여 소프트웨어 개발 산출물을 생성하는 프레임워크이다. ChatDev 는 서로 다른 역할을 맡은 에이전트의 대화를 설계, 구현, 시험과 같은 단계로 연결하는 대화형 소프트웨어 개발 프레임워크이다. 본 평가에서는 재현성을 위해 MetaGPT 0.8.2 와 ChatDev 1.1.6 의 고정 버전을 사용하며, 비교 결과는 해당 버전과 실행 조건에 한정하여 해석한다.

MetaGPT 와 ChatDev 도 역할 분담과 순차적인 개발 단계를 제공하므로, 동일한 자연어 요구사항에서 생성된 요구사항, 설계, API, 소스 코드, 테스트와 클라우드 배포 산출물이 완결되어 있는지, 서로 모순되지 않는지, 그리고 독립된 검증 절차로 실제 동작을 확인할 수 있는지를 비교한다. 각 비교 대상에는 동일한 LLM 모델과 API 엔드포인트, seed, temperature, 최대 토큰 수, 실행 시간 제한, 네트워크·Docker·Terraform 실행 환경을 적용한다. 또한, 제안하는 시스템의 구성방식을 다른 프레임워크에 강제하지 않고 비교시에 각 프레임워크의 기본 구성을 사용하도록 한다.

5.1.2. 평가 항목 및 선정 근거

비교 항목을 위해 설정한 평가 항목은 다음과 같다. 클라우드 리소스의 요구사항 만족도는 동일한 CSP, 리전, VM 스펙, 사용 비용 이 명시된 프롬프트를 기준으로, 각 개발 지원 도구가 클라우드 리소스 요구사항을 만족하는 지 여부를 파악하여 요구사항 만족도(백분율) 기준으로 도출하는 것이다. 추가적으로, 클라우드 리소스를 배포하기 위해 생성되는 IaC 코드가 정상적으로 생성되는지를 확인하였다.

표 21 은 요구사항 만족도를 평가하기 위하여 사용한 클라우드 제약사항 입력을 위한 프롬프트를 나타낸 것이다.

표 21. 비교 평가에 사용한 클라우드 제약사항

내용
This application will pilot the service on AWS in the Seoul Region, with a monthly cloud budget not exceeding USD 5. The workload must provide at least 1 vCPU, 1 GiB of memory, and 8 GiB of persistent block storage, and it must run as a single virtual-machine replica. The expected traffic pattern is steady rather than spiky. The fixed research scope is Docker on Linux virtual machines provisioned with Terraform and exposed through temporary HTTP endpoints. HTTPS, TLS certificates, domain management, managed database services, Kubernetes, and serverless compute are outside the scope of this pilot.

표 21 에 나타난 바와 같이, 사용자는 CSP, 리전, VM 지정, 월간 예산, 최소 vCPU, 최소 메모리(GiB), 트래픽 패턴, 스토리지 용량(GiB), VM 인스턴스 수에 대해 동일한 클라우드 리소스에 대한 제약사항을 입력하였다.

5.2. 평가 결과

그림 38 에서 나타난 클라우드 리소스의 요구사항 만족도는 8 가지의 비교기준으로 산정하였다. 이는 CSP, 리전, VM 지정, 월간 예산, 최소 vCPU, 최소 메모리(GiB), 트래픽 패턴, 스토리지 용량(GiB), VM 인스턴스 수의 8 가지 기준으로 비교하였다. 이 지표는 생성된 배포 산출물이 사용자가 지정한 운영 조건과 실행 환경 구성을 만족하는지를 평가하기 위한 기준이다.

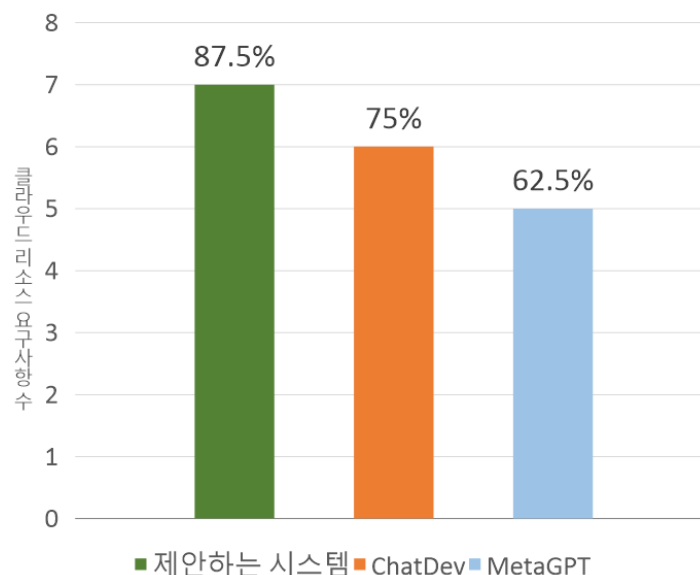


그림 38. 클라우드 리소스의 요구사항 만족도

다음은 각각 제안하는 시스템, MetaGPT, ChatDev 에 대한 IaC 의 리소스를 기준으로 월 비용을 계산한 결과이다. 클라우드 리소스 사용 비용은 Infracost 의 기본 산정 기준인 Linux/UNIX 온디맨드 요금과 월 730 시간 가동을 기준으로 계산되었다.

```

{
  "currency": "USD",
  "summary": {
    "projects": 1,
    "resources": 16,
    "costed_resources": 5,
    "free_resources": 11,
    "total_monthly_cost": "18.7236,"
  }
}

```

제안하는 시스템의 월간 비용 측정

```

{
  "currency": "USD",
  "summary": {
    "projects": 1,
    "resources": 4,
    "costed_resources": 2,
    "free_resources": 2,
    "total_monthly_cost": "8.792,"
  }
}

```

MetaGPT 의 월간 비용 측정

```

{
  "currency": "USD",
  "summary": {
    "projects": 1,
    "resources": 3,
    "costed_resources": 2,
    "free_resources": 1,
    "total_monthly_cost": "10.402,"
  }
}

```

ChatDev 의 월간 비용 측정

제안하는 시스템은 18.7236 달러, MetaGPT 는 8.792 달러, ChatDev 는 10.402 달러로 요구사항으로 설정한 월간 5 달러를 초과하여 리소스를 선택하였음을 확인하였다.

표 22 는 동일한 AWS 수강신청 앱 사례에 대해 각 도구가 생성한 IaC 의 정상 생성을 확인하기 위한 OpenTofu 기반 검증 결과를 나타낸다.

표 22. AWS 수강신청 사례의 OpenTofu 기반 IaC 검증 결과

검증 단계	제안하는 시스템	MetaGPT	ChatDev
검증 결과	통과 (종료 코드 0)	EBS 볼륨-연결-인스턴스 순환 의존성	tf 구성 에러 (선언되지 않은 변수 참조)

표 22 에 나타난 바와 같이, 제안하는 시스템은 valid: true, 오류 0 개로 검증을 통과하였다. 반면 MetaGPT 는 EBS 볼륨과 EC2 인스턴스 사이의 순환 의존성으로, ChatDev 는 선언되지 않은 입력 변수 key_pair_name 을 참조하여 각각 오류 1 개와 함께 검증에 실패하였다. 따라서 본 사례에서 제안하는 시스템이 생성한 IaC 는 OpenTofu 의 정적 구성 검증을 통과하는 수준의 유효성을 갖추었음을 확인하였다.

각 검증 결과로 나온 JSON 결과는 제안하는 시스템은 그림 39, MetaGPT 는 그림 40, ChatDev 는 그림 41 과 같다.

```
"valid": true,  
"error_count": 0,  
"warning_count": 0,  
"diagnostics": []
```

그림 39. 제안하는 시스템의 OpenTofu 검증 결과

```
"valid": false,  
"error_count": 1,  
"warning_count": 0,  
"diagnostics": [  
  {  
    "severity": "error",  
    "summary": "Cycle: aws_instance.app_instance, aws_ebs_volume.app_data",
```

그림 40. MetaGPT 의 OpenTofu 검증 결과

```
"valid": false,  
"error_count": 1,  
"warning_count": 0,  
"diagnostics": [  
  {  
    "severity": "error",  
    "summary": "Reference to undeclared input variable",
```

그림 41. ChatDev 의 OpenTofu 검증 결과

6. 결론 및 향후 연구 방향

본 연구에서 제안하는 기술은 멀티 AI 에이전트를 활용하여 순차적인 개발 단계와 사용자의 피드백을 바탕으로 클라우드 네이티브 애플리케이션 개발 전 과정을 지원한다. 본 기술은 AI 에이전트를 활용하여 사용자의 요구사항을 분석하고, 이를 바탕으로 애플리케이션의 구조와 클라우드 리소스 환경을 설계한다. 이후 설계된 내역을 바탕으로 클라우드 네이티브 애플리케이션 서비스를 구현하며, 최종 산출물에 대한 테스트를 수행하여 애플리케이션의 동작과 구성을 검증한다.

본 기술을 수강신청 애플리케이션 개발 사례에 적용하였을 때, 동일한 클라우드 제약사항을 바탕으로 클라우드 리소스와 배포 파일이 사용자의 요구사항에 맞게 구성되었는지 검증하였고 산출물들이 요구사항부터 충분히 반영하는가를 검증하였다.

소프트웨어 개발을 지원하는 멀티 AI 에이전트 프레임워크인 ChatDev 및 MetaGPT 와 비교 평가를 수행하였을 때, 본 기술은 클라우드 제약사항 8 개 중 7 개를 충족하였으며, ChatDev 는 6 개, MetaGPT 는 5 개를 충족하는 것으로 나타났다. 이를 통해 제안하는 기술이 클라우드 환경의 제약사항을 개발 과정에 반영하는 측면에서 더 우수한 성능을 보임을 확인하였다.

향후 본 기술은 클라우드 네이티브 애플리케이션 개발 과정에서 발생하는 반복적 설계·구현·검증 작업을 줄이고, 사용자의 피드백을 반영하기 위한 체계를 통해 멀티 클라우드 환경에서 배포되는 다양한 서비스 개발 지원 도구로 활용될 수 있을 것으로 기대된다.

7. 참고 문헌

- [1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, Vol. 55, No. 12, Article 248, pp. 1-38, Mar. 2023.
- [2] Cloud Native Computing Foundation, "Cloud Native Definition v1.1," Feb. 2024. [Online]. Available: <https://github.com/cncf/toc/blob/main/DEFINITION.md> (downloaded 2026, Sep. 14).
- [3] A. Richardson, "Cloud Native Reference Architecture," Cloud Native Computing Foundation, Feb. 2017. [Online]. Available: <https://www.cncf.io/wp-content/uploads/2020/08/What-is-Cloud-Native-CNCF-Webinar-23-Feb-2017-1.pdf> (downloaded 2026, Sep. 14).
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," *Proc. of the 34th Conference on Neural Information Processing Systems*, pp. 1877-1901, Jul. 2020.
- [5] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A Survey on Large Language Model Based Autonomous Agents," *Frontiers of Computer Science*, Vol. 18, No. 6, Article 186345, pp. 1-26, Mar. 2024.
- [6] LangChain, "LangGraph Overview" [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview> (downloaded 2026, Sep. 14).
- [7] OpenHands, "Software Agent SDK Architecture Overview" [Online]. Available: <https://docs.openhands.dev/sdk/arch/overview> (downloaded 2026, Sep. 14).
- [8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *Proc. of the Twelfth International Conference on Learning Representations*, Nov. 2024.
- [9] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative Agents for Software Development," *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15174-15186, Aug. 2024.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirec-

tional Transformers for Language Understanding," Proc. of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 4171-4186, Jun. 2019.

[11] I. Jacobson, M. Christerson, P. Jonsson, and G. Overgaard, Object-Oriented Software Engineering: A Use Case Driven Approach, Addison-Wesley, 1992.

[12] Object Management Group, "Unified Modeling Language (UML), Version 2.5.1" [Online]. Available: <https://www.omg.org/spec/UML/2.5.1> (downloaded 2026, Sep. 14).

[13] OpenAPI Initiative, "OpenAPI Specification, Version 3.1.1" [Online]. Available: <https://spec.openapis.org/oas/v3.1.1.html> (downloaded 2026, Sep. 14).

[14] Docker, Inc., "Docker Overview" [Online]. Available: <https://docs.docker.com/get-started/docker-overview/> (downloaded 2026, Sep. 14).

[15] HashiCorp, "What Is Terraform?" [Online]. Available: <https://developer.hashicorp.com/terraform/intro> (downloaded 2026, Sep. 14).

[16] Amazon Web Services, "Overview of Amazon Web Services" [Online]. Available: <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/amazon-web-services-cloud-platform.html> (downloaded 2026, Sep. 14).

[17] Microsoft, "Technology Choices for Azure Solutions" [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/technology-choices-overview> (downloaded 2026, Sep. 14).

[18] Google Cloud, "Products and Services" [Online]. Available: <https://cloud.google.com/products> (downloaded 2026, Sep. 14).

[19] A. Cockburn, Writing Effective Use Cases, Addison-Wesley Professional, 2000.